Security report by Claude (Fable 5.1) for Interest Protocol: the Blast launch stack at `4fe8d739`, reviewed 5 November 2026, remediation reviewed at pull request #217. Proof-of-concept tests and run logs are held by Blast beside this report's source.

Security Report: Blast launch stack

Reviewer: Claude (Fable 5.1), Anthropic's model, in an agentic security-review session for Interest Protocol (Blast). **Client:** Interest Protocol. **Date:** 5 November 2026. **Audited commit:** `4fe8d739f04bafd0775691795eb9f3727e984089` of `interest-protocol/blast-v2-contracts`. **Remediation reviewed at:** pull request #217.

Deep adversarial security review of the six launch-stack Move packages of Blast v2, in the format of Asymptotic's October 2026 launchpad report. The system map, the function inventory and the checked-and-holding record are Appendices A to C; the proof-of-concept tests and run logs are held by Blast beside this report's source.

Overview

This report covers the launch stack of ADR 0137: `blast_admin` (the shared ACL, the four delayed primitives, and one generic package version and upgrade custody), `blast_core` (basis points, apportionment, recipients and payout, the fee split, the protocol-fee ledger with the partner cut, the treasury, Merkle proofs, the migration hand-off and the owner proof), `blast_vault` (permanent LP custody for every migrated sale, its fee split, Blast's share, the partner hold and holder-reward rounds), `blast_fun` (the bonding-curve launchpad), `blast_presale` (the escrow sale) and `blast_migrators` (the Cetus and Bluefin venues). A coin is sold by one product; when the sale's target is reached the product pays Blast and the named recipients, returns a non-droppable `Migration` carrying the pool's share and the vault's sealed terms, and the next command of the same transaction passes it to the stored venue, which creates the pool, locks or burns the position and shares a vault that holds it for good.

Two immutable packages anchor the stack. `blast_admin` hands every upgradeable package one `Publication` at publish, consumed once by `wrap` into a shared `PackageVersion` (fully paused) and a shared `PackageAdmin` that seals the `Publisher` and `UpgradeCap`; every protected setting is a delayed value whose proposals wait the delay applied when they were scheduled. `blast_core` owns the money vocabulary and the three sealed hand-offs (`VenueTicket`, `Migration`, `OwnerProof`), so products and venues never import each other.

Key features

- **Custody stays in the product until migration.** Curve reserves, opening-round pots, presale escrow and every fee balance are `Balance` fields of the shared `Launch` or `Presale`; the only paths out are trades, claims, refunds and the single `migrate`, which empties them exactly (PoC-backed arithmetic in Appendix C).
- **One hand-off, one redeemer.** `Migration` has no abilities, carries the stored venue's original type name and terms sealed by the product's own `PackageWitness`, and is redeemable only by that venue; the vault admits a venue and an issuer only from one delayed list and never takes terms from a caller.
- **Delayed administration with a ceiling.** Every setting, action, allowlist and map waits the applied delay, which is at most 30 days; a delay change waits the old delay; upgrade custody starts at 48 hours; `unpause` refuses while any covered delay is zero or a change to zero is pending.
- **Exact apportionment.** Allocation plans, fee splits and presale shares divide totals by largest remainder with Blast's part rounded up and the sum preserved (300-vector property PoC).
- **Fixed supply, no mint authority after creation.** The launchpad mints exactly one billion whole tokens into a ticket and `finalize` seals the supply; the presale requires an unregulated currency with a zero-supply cap and seals the supply the same way.

Notable specifics

- **Version gates by enumeration.** Every state-mutating public function of the four upgradeable packages calls `assert_current` before any other guard, with two exceptions: `vault::share_with_position`, reachable only in the transaction of the gated `vault::new`, and `blast_vault_package::unpause`, which is this report's one finding above Informational.
- **Compiled fee maximums.** Trade fees 10 %, protocol share of a trade fee 70 %, one-time allocation 10 % per asset, LP shares 30 % per asset, partner cut 80 %, creation fee 10 SUI, sell burn 50 %, presale raise and token fees 10 %: each enforced where the value is built and, in the presale, again where it is consumed.
- **Regulated Quotes are admitted by design.** USDC on mainnet carries a deny cap; a deny-listed recipient can block a payout, a vault's collection and its re-split (accepted in the ADR 0134 amendments). Sale tokens are never regulated.
- **The migrators package has no pause bits.** Its collection and preparation calls stop only when an upgrade advances the version (accepted residual, ADR 0137 amendment).
- **Interface stubs match mainnet.** The pinned Cetus CLMM v15, Cetus LP Burn v8 and Bluefin Spot v17 signatures were verified against the live bytecode on 5 November 2026 by the repository gate and by an independent GraphQL pull.
- **Not yet published.** Every package manifest still names its own address `0x0`; the ADR 0134 stack is a fresh lineage, so the deployment runbook in `deployments/README.md` is a release condition, not a live state.

Scope

- **Project:** blast-v2-contracts (`interest-protocol/blast-v2-contracts`)
- **Branch:** `main`
- **Audited commit:** `4fe8d739f04bafd0775691795eb9f3727e984089` (PR #212, 5 November 2026 10:39 UTC), read from an isolated `git archive` snapshot.

In-scope package	Path
`blast_admin`	`sui/admin`
`blast_core`	`sui/core`
`blast_vault`	`sui/vault`
`blast_fun`	`sui/launchpad`

In-scope package	Path
`blast_presale`	`sui/presale`
`blast_migrators`	`sui/migrators`

In scope

- `blast_admin`: the ACL and root transfer, the four delayed primitives, `Publication` → `wrap`, pause bits, scheduled upgrades and immutability, Publisher borrowing.
- `blast_core`: basis points and apportionment, recipients and pay-or-vest, fee splits, the protocol-fee ledger and partner payout, the treasury, Merkle verification, the hand-off and owner proof, the seed-price math.
- `blast_vault`: the witness registry and `unpause`, vault creation and position custody, fee splitting and routing, auto-LP and buyback receipts, owner claims and split changes, Blast's per-vault share, holder-reward scheduling, publication, claims, gas settlement and closure.
- `blast_fun`: launch tickets, intake and finalization, the curve (buys, sells, the capped crossing), opening rounds, creator authority and cap reassignment, metadata, claims, migration and the owner proof, launch and runtime configuration, `unpause`.
- `blast_presale`: creation and settings, rate changes, the three commitment paths with caps, phases and hard caps, settlement and early close, cancellation, claims and vesting, migration and the owner proof, creation and fee policies, `unpause`.
- `blast_migrators`: Cetus and Bluefin preparation, migration, collection, settled split updates, buybacks, the venue configuration, and the fidelity of every external function the two venues call against mainnet bytecode.
- The seams: core → vault → venues, launchpad → core → migrators, presale → core → migrators, `blast_admin` delayed modules → every consumer, partner cut → every ledger payout.

Out of scope

- `blast_partners`, `blast_sui`, `boosts/*`, `otc`, `vesting`, `coin_template`, `launch_ticket_template`, `presale_testkit`, `vendors/*` and the `interfaces/*` stubs as logic. This report makes no statement about them beyond what Trusted Dependencies relies on.
- The Rust PTB harness and the venue fork gate (`sui/launchpad/ptb_tests`), the SDK, blast-infra decoders, the admin console and off-chain signers.
- Formal verification; this review tests, it does not prove.

Dependencies: `blast_partners`, `blast_fun_vesting`, the generated coin packages, the Sui Framework and Move Standard Library at the pinned revision, and the three external venue packages for interface fidelity. Trusted Dependencies sets out what each is trusted for.

Test suites run (`sui move test --build-env mainnet --threads 1`, CI fixture addresses, one package per heavy slot): admin 198/198, core 79/79, vault 120/120, launchpad 444/444, presale 282/282, migrators 40/40 (1,163 stock tests). PoC tests written for this audit: core 3/3, vault 3/3, launchpad 3/3. Repository gates: `check_move_style.py` (233 modules), `check_move_errors.py` (337 errors with exact-abort oracles), `check_admin_delay.py`, every package's `check_source_boundaries.sh`, `check_mainnet_abi.mjs` self-test 74/74 and live PASS.

Summary of Results

Severity	Confirmed findings
Critical	0
High	0
Medium	0
Low	1
Informational	4

Hypotheses (not counted above): 1.

No finding lets an untrusted party take, redirect or permanently lock funds. The Low concerns upgrade hygiene: the vault's `unpause` wrapper is the only state-changing public function in the four upgradeable packages that skips the version gate, so after a future upgrade an `UnpauseAdmin` could release the vault through superseded bytecode that enforces an older, weaker gate. The Informational findings concern a release gate that does not reach the core treasury's delay, an event that omits the vesting schedule its presale twin carries, holder-reward payouts that strand at object addresses named by the off-chain manifest, and the coupling of auto-LP compounding to permissionless fee collection. Everything attacked under the §4 catalogue and found to hold is recorded in Appendix C.

Remediation. The client fixed all five findings in pull request #217 (branch `claude/deep-audit-fixes`, head `0078f8fa`, rebased on `main` at `88951fe9`). Each fix was checked against its finding; at that head the six package suites (1,167 tests, six of them new) and every repository gate pass. H-1 remains a hypothesis: a venue refusal of the auto-LP add can no longer block collection, and the launchpad fork gate exercises `compound_auto_lp` against Cetus and Bluefin bytecode after each collection; adversarial pool states stay open as blast-v2-contracts#218.

Findings

ID	Title	Severity	Status
L-1	The vault's `unpause` is not version-gated, so superseded bytecode can release pause bits after an upgrade	Low	Remediated
I-1	The core `Treasury` delay is outside every release gate; one visible 48-hour wait turns redirection into a one-call setter	Informational	Remediated
I-2	`MigrationAllocationPaid` omits the vesting schedule ID that the presale's allocation events carry	Informational	Remediated
I-3	Holder-reward claims are relayable transfers to the leaf's beneficiary, so a leaf naming an object address strands its payout	Informational	Remediated
I-4	Auto-LP compounding runs inside permissionless fee collection, so any venue-side abort in the add blocks that vault's collection	Informational	Remediated

L-1 · LOW · Remediated

The vault's `unpause` is not version-gated, so superseded bytecode can release pause bits after an upgrade

Location: `blast_vault`, `sui/vault/sources/blast_vault_package.move:148-158`; compare `sui/launchpad/sources/launch_config.move:383-399`, `sui/presale/sources/creation_policy.move:180-196` and the vault's own `make_immutable` at `blast_vault_package.move:163-172`.

Description. `blast_vault_package::unpause` checks `self.witnesses.is_guarded()` and then calls the generic `package_admin::clear_pause_bits`, which by design reads no consumer version (`sui/admin/sources/package_admin.move:254-269`). Neither call compares the shared `PackageVersion<BLAST_VAULT_PACKAGE>.version` with the bytecode's compiled `constants::version!()`. The launchpad's and the presale's `unpause` wrappers, and the vault's `make_immutable` wrapper two functions below, all begin with `package_version.assert_current(constants::version!())`. On Sui a superseded package version stays callable and shares every type with its successors, so once `commit_upgrade_and_advance` moves the version to 2, version-1 bytecode's `unpause` still accepts the live `PackageVersion`, still passes the version-1 guard, and clears the requested bits.

Impact. A future vault release that tightens its release gate, for example by covering a new delayed setting, cannot stop an `UnpauseAdmin<BLAST_VAULT_PACKAGE>` from releasing the vault through the version-1 wrapper, which only checks the witness-admission delay. The role is required, so no untrusted party can trigger it; the package is unpublished and no second version exists, so nothing is exposed today. This is the Asymptotic L-5 class: a guard the package's own convention calls for is missing on one function.

Recommendation. Add `package_version.assert_current(constants::version!());` as the first statement of `blast_vault_package::unpause`, matching the product wrappers, and add the test the launchpad and presale already have (`stale_version_is_rejected_before_the_delay_gate`) to `blast_vault_package_tests.move`: set the version to `constants::version!() + 1` and expect `blast_admin::errors::EVersionMismatch` at `blast_admin::package_version`. Body-only change; the signature stays.

Evidence. the evidence bundle's `poc/vault/audit_poc_vault_tests.move`. `audit_poc_stale_version_unpause_still_clears_bits` pauses the admin bit, sets the stored version to 2 and clears it through `unpause` from version-1 bytecode; two controls show the same stale bytecode's `schedule_witness_delay` and `make_immutable` abort with `EVersionMismatch`.

```
cd sui/vault && sui move test --build-env mainnet --threads 1 audit_poc
[ PASS    ] blast_vault::audit_poc_vault_tests::audit_poc_stale_version_gated_registry_call_is_rejected
[ PASS    ] blast_vault::audit_poc_vault_tests::audit_poc_stale_version_make_immutable_is_rejected
[ PASS    ] blast_vault::audit_poc_vault_tests::audit_poc_stale_version_unpause_still_clears_bits
Test result: OK. Total tests: 3; passed: 3; failed: 0
```

Log: `logs/poc-vault.log`.

Remediation note. Pull request #217 adds `package_version.assert_current(constants::version!())` as the first statement of `blast_vault_package::unpause`, with the test `unpause_rejects_a_stale_version`; `scripts/check_admin_delay.py` now pins the wrapper beside the products' own.

I-1 · INFORMATIONAL · Remediated

The core `Treasury` delay is outside every release gate; one visible 48-hour wait turns redirection into a one-call setter

Location: `blast_core`, `sui/core/sources/blast_treasury.move:32-47`; the gates at `sui/launchpad/sources/launch_config.move:393-396` and `runtime_config.move:173-175`, `sui/presale/sources/creation_policy.move:190-193`, `sui/vault/sources/blast_vault_package.move:155`.

Description. The treasury destination is a `Delayed<DestinationKey, address>` compiled at 48 hours, so no setup step can leave it at zero. `schedule_destination` takes the next `delay_ms` from the `TreasuryAdmin`, and `delayed::schedule` accepts any value up to 30 days, zero included; the proposal waits the applied 48 hours. Once applied at zero, every later `schedule_destination` and `apply_destination` pair runs in one transaction. The pre-audit's LP-L1 fix made each product's `unpause` assert that the notices it covers are guarded; at this commit the treasury moved to `blast_core`, and no product's or the vault's gate reads `Treasury::is_guarded`. The ADR 0134 hardening amendment chose "nonzero, not a fixed 48-hour floor, and no permanent floor", so a reduction is a visible, cancellable proposal; nothing in code, however, flags a treasury that has already reached zero.

Impact. A `TreasuryAdmin` whose key is compromised needs one uncancelled 48-hour proposal, after which Blast's fees from every launch, presale and vault go to an address it chooses in the same call as the redirect; `claim_protocol_fees`, `pay_protocol_fees_with_partner` and every vault collection read the destination live. `CancelAdmin` can cancel the first proposal, and the monitoring the pre-audit asked for (`DelayedScheduled.delay_ms == 0`) catches it. No untrusted party can trigger anything.

Recommendation. Either have `blast_treasury::schedule_destination` reject a zero next delay (a floor only for this one setting, which the owner may decline per ADR 0134), or make the three `unpause` wrappers and the vault's also assert `treasury.is_guarded()`, and add a test that `unpause` refuses while the treasury delay is zero. Record the chosen answer in ADR 0134.

Evidence. `poc/launchpad/audit_poc_launchpad_tests.move::audit_poc_release_gate_does_not_cover_the_treasury_delay` lowers the delay to zero through one 48-hour wait, redirects to `0xBEEF` in one call, and shows `launch_config::unpause` still clearing every bit. Log: `logs/poc-launchpad.log`.

Remediation note. Pull request #217 makes every `unpause` (vault, launchpad, presale) take the shared `blast_core::blast_treasury::Treasury` and assert `treasury.is_guarded()` beside the package's own notices, with one zero-treasury-delay test per package. ADR 0134's amendment of 5 November 2026 records the choice of a gate over a floor.

I-2 · INFORMATIONAL · Remediated

`MigrationAllocationPaid` omits the vesting schedule ID that the presale's allocation events carry

Location: `blast_fun`, `sui/launchpad/sources/launch.move:858-868` and `sui/launchpad/sources/lib/events.move:198-210,372-378`; compare `sui/presale/sources/events.move:123-144`.

Description. ADR 0137 owner decision 6 let launchpad recipients vest. `launch::migrate` calls `payout::pay_or_vest` for each recipient's Meme and Quote and discards the `Option<ID>` it returns, then emits `MigrationAllocationPaid { launch_id, destination, meme_amount, quote_amount }`. The presale's `RaiseAllocated` and `RecipientAllocated` carry `vesting_id: Option<ID>` for the same primitive. A vested recipient's coins leave the launch into one or two shared `Vesting` objects created in the same transaction; the launchpad event says they were paid.

Impact. Indexing only. The vesting package emits `VestingCreated { vesting_id, coin_type, beneficiary, total_amount, … }` in the same transaction, so a reducer can recover the fact by joining on transaction, beneficiary and amount; a reducer that reads the launchpad stream alone will show vested recipients as paid. The lineage is unpublished, so the schema can still change at no cost.

Recommendation. Add `meme_vesting_id: Option<ID>` and `quote_vesting_id: Option<ID>` to `MigrationAllocationPaid` and emit the two IDs `pay_or_vest` returns, as the presale does; pin them in the existing migration-vesting test (`migration_vests_a_vesting_recipients_allocation`).

Evidence. By reading; the emitter signature at `events.move:198-204` takes no schedule ID.

Remediation note. Pull request #217 adds `meme_vesting_id` and `quote_vesting_id: Option<ID>` to `MigrationAllocationPaid`, emitted from the values `pay_or_vest` returns and pinned in `migration_vests_a_vesting_recipients_allocation`.

I-3 · INFORMATIONAL · Remediated

Holder-reward claims are relayable transfers to the leaf's beneficiary, so a leaf naming an object address strands its payout

Location: `blast_vault`, `sui/vault/sources/holder_rewards.move:193-243`; `sui/core/sources/payout.move:103-112`.

Description. `claim` verifies the leaf `(vault, round, index, beneficiary, meme, quote)`, marks its bit, and pays with `payout::pay`, a `transfer::public_transfer` of a fresh coin to `beneficiary`. Anyone may relay a proof. The round's leaves come from an off-chain allocation of checkpointed holder balances; some of those balances belong to objects, not accounts (DEX pools, vesting schedules, other vaults). A coin transferred to an object's address can be recovered only by that object's module through `transfer::receive`; a Cetus pool or a `Vesting` object has no such path. The claim succeeds, the bit is set, and the Meme and Quote are gone; `close_round` cannot return them because the leaf is marked claimed.

Impact. Bounded by the leaf amounts the manifest assigns to object addresses; no third party gains. A griefer who wants to burn those rewards only has to relay the proof before the deadline, which is permitted by design. The control is off chain: the manifest must exclude object-held balances or route them to an account.

Recommendation. Document the exclusion in the holder-rewards manifest specification and the `RewardsAdmin` runbook, and have the off-chain checker refuse a leaf whose beneficiary is a known object (any address that resolves to an object at the checkpoint). On chain nothing cheap distinguishes the two.

Evidence. By reading: `holder_rewards.move:231-232` transfers unconditionally; `vault.move`/`holder_rewards.move` offer no receive path.

Remediation note. Pull request #217 documents in `holder_rewards` and the vault integrator guide that leaves name accounts; the off-chain allocation's exclusion of object-held balances is tracked as blast-v2#723.

I-4 · INFORMATIONAL · Remediated

Auto-LP compounding runs inside permissionless fee collection, so any venue-side abort in the add blocks that vault's collection

Location: `blast_migrators`, `sui/migrators/sources/cetus.move:320-342,378-441` and `sui/migrators/sources/bluefin.move:300-313,334-361,389-426`.

Description. Both venues' `collect_fees` and `settle_and_apply_fee_split` call `settle_fees`, which collects, splits through the vault and then `compound`s: it takes the whole auto-LP balance with a receipt, chooses a side, calls the venue's add-liquidity, and returns the rest. The side choice is meant to guarantee the venue never needs more of the other asset than the vault holds. Cetus's choice calls Cetus's own `clmm_math::get_liquidity_by_amount` with the same inputs the add uses, so it agrees by construction. Bluefin's choice re-derives the venue's formulas locally with a two-atom margin (`can_fix_a`, `can_fix_b`). If the venue ever asks for more than the local estimate plus the margin, `add_liquidity_with_fixed_amount` aborts in the venue, and because compounding is on the collection path, every `collect_fees` and `settle_and_apply_fee_split` on that vault aborts until the auto-LP balance changes, which only a collection can do. The owner's fee split cannot be changed to drop the auto-LP route either, because applying a split also settles first.

Impact. No funds move; a vault's fees stay in the position until a migrators upgrade (48-hour notice) adds a path that collects without compounding. The trigger is not demonstrated (see H-1) and the stock suite's grid and extreme-value tests of the Bluefin side selection pass. A Cetus or Bluefin pause has the same effect through `collect_fee` itself and is an accepted venue-operator lever (MG-I1).

Recommendation. Keep compounding off the collection path: either make `compound` its own permissionless call, or have `collect_fees` skip the add and leave the balances waiting when the venue's own preview says the add would not fit. Add a test that a collection succeeds when the auto-LP balances cannot be added.

Evidence. By reading; the control flow at `cetus.move:332-341` and `bluefin.move:310-312` is unconditional.

Hypotheses

ID	Statement	Why it could not be proven
H-1	Bluefin's `add_liquidity_with_fixed_amount` can require more of the other asset than `auto_lp_fixed_side` estimates plus two atoms for some pool state, which through I-4 would stop that vault's collection.	The venue's pool bytecode cannot execute locally (the interface is `abort 0` stubs); the Rust fork gate that runs real venue bytecode needs a full simulator build that the host's disk and memory budget did not allow in this session.

Verification

This audit tests; it does not prove. For each package, the invariants from the system map and how each is covered: by a stock test (S), by a PoC written for this audit (P), by reading only (R), or not at all (–).

Package	Invariant	Coverage
admin	one `Publication` per package, consumed once by `wrap` with a version-1 cap	S
admin	version moves only by +1 inside `commit_upgrade_and_advance`	S
admin	proposals wait the applied delay; a shorter next delay waits the old one; ≤ 30 days	S
admin	`take_action`/`apply_delay` reject the other proposal kind	S
admin	unpause needs the package witness and a guarded upgrade delay	S
admin	a revoked or destroyed cap signs in to nothing; root transfer waits 7 days	S
core	apportionment sums to the total and the rounded-up part gets its ceiling	S, P
core	ledger pays only the treasury, partner first when bound, destroyed only empty	S
core	hand-offs cannot be dropped; `redeem` requires the stored venue; `new` requires the sealing witness	S

Package	Invariant	Coverage
core	seed price is the exact floor square root; seedable bounds reciprocal	P, S
core	Merkle leaves and nodes are domain-separated; depth and index exact	S, P
vault	venue and issuer both admitted; one vault per (issuer, origin); positive liquidity	S
vault	collection conserves gross = cut + Σ routes; nothing left over	S
vault	receipts bind the vault and cap returns	S
vault	rounds: number sequential, interval non-overlapping, budget within rewards, bitmap once, deadline recovery	S
vault	owner actions need a proof for the exact terms; 48-hour fixed split and share delays	S
vault	every state-mutating function is version-gated	P (L-1 found)
launchpad	one launch per Meme; pair distinct; Quote, venue, decimals admitted; supply exact; sell burn ≤ 50 %	S
launchpad	capped net input equals the gap; `reaches_target` ⇔ `real == target`	P, S
launchpad	buy-then-sell and split buys never profit; sells never overdraw the real reserve	P, S
launchpad	opening-round pot and bids move together; delivery exact, last takes remainder	S
launchpad	`migrate` empties every balance and pays exact shares	S
launchpad	creator claim one-time, bound to sender or signed message, caps honoured by ID	S
launchpad	release gate covers every launchpad delay	S; treasury not covered (I-1, P)
presale	one presale per coin; unregulated, unknown-supply currency; supply sealed	S
presale	shares sum exactly; LP share positive; seedable at creation and after rate changes	S
presale	commitments respect caps, minimums, participants, hard cap	S
presale	claims pro rata, last takes remainder; cancellation refunds all	S
presale	`migrate` moves exactly the target and the LP tokens	S
migrators	only the stored venue redeems; pair exact; Cetus order canonical	S (unit), R (venue)
migrators	Cetus creation reserved before Meme leaves custody; no adoption	R; fork gate not run
migrators	collection touches only the vault's pool and routes through `split_fees`	R
migrators	auto-LP add never overdraws	S (side selection), – (venue execution; H-1)

Coverage matrix (package × §4 subsection): a finding ID, `Clean` (attacked, held; details in Appendix C) or `N/A` with a reason.

Package	4.1 Objects	4.2 PTBs	4.3 Arithmetic	4.4 DEX	4.5 Authority	4.6 Protocol	4.7 Bounds	4.8 Events	4.9 Tests
admin	Clean	Clean	Clean (deadline math)	N/A: no venue	Clean	N/A: no economics	Clean (roles ≤ 32)	Clean	Clean
core	Clean	Clean	Clean	N/A: price math only, no venue call	I-1	Clean (ledger, Merkle)	Clean (routes ≤ 20)	Clean	Clean
vault	Clean	Clean	Clean	Clean (receipts)	L-1	I-3	Clean (leaves ≤ 65,536)	Clean	gap: stale-version unpause test
launchpad	Clean	Clean	Clean	N/A: no venue call (hand-off only)	Clean	Clean	Clean (recipients ≤ 16)	I-2	gap: seedable-price pin
presale	Clean	Clean	Clean	N/A: hand-off only	Clean	Clean	Clean (participants ≤ 65,536, recipients ≤ 64)	Clean	Clean
migrators	Clean	Clean	Clean (Bluefin u256 math)	I-4, H-1; fidelity Clean	Clean (no pause by design)	N/A: no sale logic	Clean	Clean	gap: venue behaviour only in the fork gate

Trusted Dependencies

Dependency	Kind	Address
`blast_partners`	package (local, immutable)	— (unpublished lineage)
`blast_fun_vesting`	package (local, immutable)	—
generated coin packages (`launch_ticket_template`, presale creators' coins)	packages	—
Sui Framework	framework, rev `06734f6ff0af45d8632a14a4dc4b100197f6b1a2`	`0x0000000000000000000000000000000000000000000000000000000000000002`
Move Standard Library	framework, same rev	`0x0000000000000000000000000000000000000000000000000000000000000001`
Cetus CLMM v15	external package	original `0x1eabed72c53feb3805120a081dc15963c204dc8d091542592abaf7a35689b2fb`, current `0x260693ec785a6e6c9d81d58c7d2ff72f1288ae0fa6a9725abe05a6478b11f084`
Cetus LP Burn v8	external package	original `0x12d73de9a6bc3cb658ec9dc0fe7de2662be1cea5c76c092fcc3606048cdbac27`, current `0xc4aabc874f5060d3147d8f74c14e3a8a1f9f8d92edea138a51635395ed167cd9`

Dependency	Kind	Address
Bluefin Spot v17	external package	original `0x3492c874c1e3b3e2984e8c41b589e642d4d0a5d6459e5a9cfc2d52fd7c89c267`, current `0xd075338d105482f1527cbfd363d6413558f184dec36d9138a70261e87f486e9c`
`integer_mate` v7	external package	`0x714a63a0dba6da4f017b42d5d0fb78867f18bcde904868e51d951a5a6f5b7f57`

The stack's guarantees hold only relative to the dependencies below, which this review takes as correct.

Dependency	Trusted for
`blast_partners`	`Partner` objects are shared and never destroyed, so a bound `partner_id` always resolves; `pay` sends at most the applied share, rounded down, to the cap-set recipient and leaves the rest; shares ≤ 8,000 bps change only after a fixed 7-day delay; `PartnerList` accepts a partner at once and keeps a removed one accepted for that delay
`blast_fun_vesting`	`new_irrevocable` locks the whole coin for the named beneficiary from `start_ms = now` with the given cliff and steps, rejects a zero beneficiary and a past start, and lets only the beneficiary claim vested value; `VestingCreated` carries the schedule ID, beneficiary and amount
generated coin packages	the `launch_ticket_template` initializer passes its one-time witness and the creator's choices to `launch_ticket::new` and delivers the ticket and `TreasuryCap` to the publisher; presale creators' coins are registry currencies whose `TreasuryCap` holder is the sale's creator
Sui Framework	conservation of `Balance` and `Coin`; Coin Registry supply modes (`make_supply_fixed`, `make_supply_burn_only`, `burn_balance`), metadata caps and the `Currency` BCS layout; dynamic fields and dynamic object fields; derived object IDs (one claim per key); package upgrade tickets, receipts, policies and `make_immutable`; Ed25519 verification; the clock; shared-object deletion; transfers and `public_transfer` deny-list enforcement
Move Standard Library	type identities through `type_name` (`with_original_ids`, `with_defining_ids`, `original_id`, `is_primitive`); checked u64/u128/u256 arithmetic, `mul_div`, `mul_div_ceil`, `div_ceil`, `checked_add`; options, vectors, BCS
Cetus CLMM v15	`get_fee_rate` aborts for an unregistered spacing; `mint_pool_creation_cap` mints one cap per coin type from its `TreasuryCap`; `register_permission_pair` makes the pair's creation at that spacing permissioned; `create_pool_v3_with_creation_cap` creates exactly one pool at the given price and seeds the full range from the coins passed, returning the position and residuals; `add_liquidity_fix_coin`/`repay_add_liquidity` pay exactly `add_liquidity_pay_amount`; `flash_swap`/`repay_flash_swap` by amount in; `clmm_math::get_liquidity_by_amount` is the function the add path uses
Cetus LP Burn v8	`burn_lp_v2` permanently locks the position's liquidity and returns a proof that `collect_fee` can use to collect the position's fees and nothing else
Bluefin Spot v17	`create_pool_and_get_object` creates a fresh pool for exactly the configured SUI fee at the given price and tier; `open_position` and `add_liquidity_with_fixed_amount` return the unused balances; `collect_fee` and `swap` as named; the pool manager role is limited to reward emissions (ADR 0088)
`integer_mate`	`i32` arithmetic for full-range tick alignment

Two dependencies carry the most weight. Every role-gated result assumes the ACL hands a role's witness only to a live `AdminCap` holding that role, so access control is exactly as strong as the ACL and the custody of its root. Every venue result assumes the external packages behave as their verified signatures suggest; the fork gate, not this review, is the local evidence for their behaviour.

Coverage and Limitations

This report establishes properties of the six packages at the audited commit relative to the dependencies above, by source review and local tests. It does not establish the following.

- **Venue execution.** No Cetus or Bluefin bytecode ran in this review; the interface stubs are `abort 0`. Pool creation, seeding, burning, fee collection, auto-LP adds and swaps are covered only by the repository's fork gate at its last recorded run, which this session could not build (the `ptb_tests` simulator needs more disk and memory than the shared host budget allowed on 5 November). H-1 and the venue rows of the verification table depend on it.
- **PTB-level behaviour.** Result handling, `entry` restrictions and multi-command flows were traced by reading, not executed. Every PoC is a Move unit test.
- **Interface fidelity scope.** Signatures, type parameters, abilities, type-argument order and module fingerprints were compared against mainnet; behaviour was not. A venue upgrade after 5 November invalidates the comparison.
- **Upgrades.** Old bytecode stays callable forever; a governed upgrade under the compatible policy can replace any body in the four upgradeable packages after its notice. `blast_admin`, `blast_core` and `blast_partners` are to publish immutable, so a defect in them needs a fresh lineage.
- **Outside the packages.** Root, upgrade, treasury and claim-signer key custody; the off-chain reward manifests (I-3); the SDK and admin console; blast-infra, whose `origin/main` decoders (`7c52521`, 4 November) still target the contracts #168 layouts and not the #198–#212 moves of `ProtocolFeesPaid` to `blast_core::blast_core_events`, the vault events to `blast_vault::blast_vault_events` and the venue events to `blast_migrators::events`. That gap is filed as blast-infra#126; this report makes no statement about replay completeness downstream.
- **Deny lists and pauses of regulated Quotes**, a Cetus pool-manager override, and venue version bumps are accepted trust boundaries (ADR 0134 amendments) and were re-read, not re-rated: the code at this commit does not make any of them worse than the ADR describes.
- **Budget.** All `sui move test` runs used one thread. No coverage report was generated.

Methodology

Review. Each of the 15,112 production lines of the six packages was read line by line in dependency order (admin, core, vault, launchpad, presale, migrators), against the ADRs (0134 and its amendments, 0137 and its 2026-11-05 amendment, 0088, 0097, 0114, 0117), the two prior reviews (Asymptotic October 2026 at `01d76596`; the 3 November pre-audit at `b841a81` and its fixes in #167/#168) and the PR150 reviews. Every prior fix was re-located at this commit before anything else was reported; accepted residuals were re-read for regressions and are not re-reported.

System map. Every shared, owned, capability, hot-potato and witness type was tabulated with its abilities and lifecycle; SUI, Quote and Meme were traced from entry to every exit; the invariants each package must hold were written down before the hunt (Appendix A).

Inventory. Every `public`, `public(package)` and `entry` function (593) was extracted mechanically with its version gate, pause gate, authority parameter, value flow, emitted events and rounding primitives, then checked by hand (Appendix B).

State tables. The launch, presale, vault round slot, delayed-primitive and package-version state machines were tabulated function by state and compared with the code.

Adversarial catalogue. Every item of the brief's §4 was applied to every package it could apply to; the results, including negative ones, are in Appendix C.

Proof of concept. A finding is **confirmed** only when a Move test on the audited code demonstrates it (L-1) or when the mechanism is a direct, unconditional reading of the code with no behavioural assumption (I-2, I-3, I-4). I-1 is confirmed by a test of the treasury and gate behaviour. A claim whose trigger could not be executed is a **Hypothesis** and is listed separately. Properties the review relied on were also turned into tests (apportionment, capped crossing, round trips, square root, Merkle negatives), all of which pass.

Gates. The repository's style, error-registry, administration-boundary and source-boundary gates and the live ABI gate were run on the snapshot; all pass.

Commits

Revision	Commit
Audited commit (`origin/main`, PR #212)	`4fe8d739f04bafd0775691795eb9f3727e984089`
Test logs and PoCs	same commit, isolated snapshot with CI fixture addresses
Paused deep-audit pass (superseded)	`61e159ce25aaa654f990fd9b340a581aa5283757`
Pre-audit (3 November 2026)	`b841a81`; fixes `2de44b6` (#167), `9d768848` (#168)
Asymptotic final reviewed commit	`01d7659617abcd552cece6d05b9db409ec4ff2c0`
Sui Framework pin	`06734f6ff0af45d8632a14a4dc4b100197f6b1a2`
Venue packages verified live	Cetus CLMM v15 `0x260693ec…11f084`, Cetus LP Burn v8 `0xc4aabc87…167cd9`, Bluefin Spot v17 `0xd075338d…486e9c` (5 November 2026)
blast-infra decoders read	`7c52521` (`origin/main`, 4 November 2026)
Remediation (pull request #217)	`e2139835` fixes L-1, I-1, I-2, I-3, I-4; `0078f8fa` links the record; rebased on `main` `88951fe9`

This report reflects a review of the source and local tests at the commits noted above, conducted by Claude (Fable 5.1), an AI model, in an agentic session for Interest Protocol. A review reduces the risk of certain classes of bugs but does not guarantee the absence of all vulnerabilities; only the reviewed packages and properties are covered. It is not a formal proof. This report is provided for informational purposes only, not as financial, legal, or investment advice. No liability is assumed for losses arising from reliance on this report. Users should perform their own due diligence before interacting with any smart contract system.

Remediation note. Pull request #217 makes compounding its own permissionless route, `compound_auto_lp`, on Cetus and Bluefin; `collect_fees` and `settle_and_apply_fee_split` only collect and split, and Cetus `collect_fees` drops its unused `Clock`. The migrators source-boundary gate pins the new shape. The launchpad fork gate appends `compound_auto_lp` to each collection against Cetus and Bluefin bytecode; adversarial pool states for H-1 stay open as blast-v2-contracts#218.

Appendix A. System map

Objects, capabilities, value flows, invariants and state tables for the six launch-stack packages at `4fe8d739f04bafd0775691795eb9f3727e984089`. Line references are to the audited snapshot.

1. Objects, capabilities, witnesses and hot potatoes

blast_admin (immutable)

Type	Abilities	Created by	Held by	Mutated by	Destroyed by
`ACL`	key	`blast_acl::init` (one)	shared	root: `grant_admin`, `revoke_admin`, `grant_role`, `revoke_role`, transfer flow; anyone: `destroy_admin_cap`, `accept_super_admin_transfer` (nominee)	never
`SuperAdminCap`	key	`init` → `@blast_acl_root`; `accept_super_admin_transfer` mints the successor	one address, no `store`	—	`destroy_retired_` (only an inert one)
`AdminCap`	key, store	`grant_admin`	any address (transferable)	—	`destroy_admin_ca`
`AdminWitness<Role>`	drop	`sign_in<Role>` against the live ACL	transaction-local	—	dropped
`Publication<OTW>`	key	`package_version::new_publication` in each consumer `init`	deployer	—	consumed by `packag` only
`PackageVersion<OTW>`	key	`wrap` (one per OTW)	shared	`set_pause_bits` (PauseAdmin via `pause`), `clear_pause_bits` (UnpauseAdmin through the package's own `unpause`), `advance` (through `commit_upgrade_and_advance`)	never
`PackageAdmin<OTW>`	key	`wrap`	shared; seals `Publisher` and `Option<UpgradeCap>`	UpgradeAdmin: schedule/authorize/commit/immutable/delay; CancelAdmin: cancel	never
`Delayed<K,T>`, `DelayedAction<K,A>`, `DelayedSet<K,E>`, `DelayedMap<K,E,V>`	store	owner module with its `&UID`	inside the owner object	owner module only	`destroy` with nothing drops its table)

Type	Abilities	Created by	Held by	Mutated by	Destroyed by
`UpgradeTicket` / `UpgradeReceipt`	framework hot potatoes	`authorize_upgrade` / native `Upgrade`	PTB	—	native upgrade / `commit_upgrade_a`

blast_core (immutable)

Type	Abilities	Notes
`Treasury`	key	one shared destination, `Delayed<DestinationKey, address>` at a compiled 48 h; TreasuryAdmin schedules, CancelAdmin cancels, anyone applies
`Ledger<Meme, Quote>`	store	Blast's Meme, Quote and SUI held by a launch, presale or vault; created only with a `PackageWitness`; bound to one `partner_id` for good; `pay` aborts when a partner is bound, `pay_with_partner` requires that exact `Partner`
`Bps`	copy, drop, store	0–10,000, `new` is the only constructor
`FeeSplit`, `Route`, `Destination`	copy, drop, store	≤ 20 distinct routes summing to 10,000; address routes reject `0x0`
`Recipient`, `VestingTerms`	copy, drop, store	nonzero destination; vesting steps fit `2^63` ms
`VaultTerms`	copy, drop, store	origin, sealing witness (original ids), split, Blast shares ≤ 3,000 bps, partner, residual policy; public constructor needs a witness value
`VenueTicket<Meme, Quote>`	none	issued by a venue with its witness; consumed by a product's `new`
`Migration<Meme, Quote>`	none	pool Meme and Quote, stored venue, terms; created only with the product witness that sealed the terms; redeemed only with the stored venue's witness
`OwnerProof`	none	sealing witness and origin; consumed by `assert_terms` against a vault's terms
product roles	no abilities (type-level)	`TreasuryAdmin`, `FeeAdmin`, `ListingAdmin`, `VenueAdmin`, `RewardsAdmin`, `BuybackAdmin`, `CreatorAdmin`, `ClaimSignerAdmin`, `CancelAdmin`

blast_vault (compatible-upgradeable, immutable after audit)

Type	Abilities	Notes
`VaultRegistry`	key	one shared; `DelayedSet<WitnessesKey, TypeName>` of admitted venue and issuer witnesses (starts at zero delay); its UID derives every vault's ID
`Vault<Meme, Quote>`	key	derived from `VaultKey { package_witness, origin }`; terms, `pool_id`, `Delayed` fee split and Blast share (fixed 48 h), protocol `Ledger`, seven balances, round cursor, `DelayedAction<RoundPublicationKey, RoundTerms>` (2 h)
venue position	key + store, under `PositionKey<Venue>`	dynamic object field; borrowed only by the venue's witness; never removed
auto-LP position	key + store, under `AutoLpPositionKey<Venue>`	attached once by the venue; never removed
`Round<Meme, Quote>`	key	shared at publication; funded claims, bitmap, frozen treasury, optional gas reserve; deleted by `close_round`
`VaultWitness`	drop	creates the vault's `Ledger`; constructed only inside `vault`
`BuybackReceipt`, `AutoLpReceipt`	none	bind a venue's take to the same vault and cap what returns
`PackageWitness`	drop	pinned at publication; only `blast_vault_package` constructs it

blast_fun (compatible-upgradeable)

Type	Abilities	Notes
`LaunchRegistry`	key	id-only; `LaunchKey<Meme>` derives the one launch per Meme
`LaunchConfig`	key	decimals, Quotes, `Delayed` trade fees, `DelayedSet` venues, `Delayed` protocol allocation, LP-fee share, creation fee (all zero delay at publication), `PartnerList`
`RuntimeConfig`	key	`DelayedSet` claim-signer keys (≤ 4 live + pending), `Delayed` protocol fee share ≤ 7,000 bps (zero delay at publication)
`LaunchTicket<Meme>`	key, store	supply, `MetadataCap`, creator, decimals, burnable, sell-burn ≤ 5,000; the `TreasuryCap` travels beside it
`LaunchInitializer<Meme, Quote>`	none	hot potato from `new` to `finalize`
`Launch<Meme, Quote>`	key	derived ID; lifecycle, opening round, curve reserves, `pending_burn`, allocations, protocol `Ledger`, creator fee balances, native `MetadataCap`, two `DelayedAction` cap reassignments (48 h), creator recipient, live cap IDs, partner
`CurveFeeCap<Meme>`, `CreatorMetadataCap<Meme>`	key, store	honoured only while their ID is the one the launch records
`CreatorRecipient`	copy, drop, store	address or 32-byte account commitment
`PackageWitness`	drop	only `blast_fun::blast_fun` constructs it

blast_presale (compatible-upgradeable)

Type	Abilities	Notes
`PresaleRegistry`	key	`PresaleKey<SaleToken>` derives the one presale per coin

Type	Abilities	Notes
`CreationPolicy`	key	Quotes, `DelayedSet` venues (zero delay at publication), `PartnerList`
`FeePolicy`	key	`Delayed` raise ≤ 1,000, token ≤ 1,000, LP shares ≤ 3,000 bps (zero delay at publication)
`PresaleInitializer<SaleToken, Quote>`	none	hot potato from `new` to `finalize`; holds the `TreasuryCap`
`Presale<SaleToken, Quote>`	key	derived ID; lifecycle, windows, eligibility, uncapped phase, hard cap, venue, split, commitments (dynamic fields), escrowed token balances, payouts, protocol `Ledger`
`PresaleCap`	key, store	bound to one `presale_id`
`PackageWitness`	drop	only `blast_presale_package` constructs it

blast_migrators (compatible-upgradeable, no pause bits)

Type	Abilities	Notes
`VenueConfig`	key	write-once dynamic fields: Cetus `PoolCreationCap` (object field) and tick spacing per Meme, Bluefin tier per Meme
`cetus::Venue`, `bluefin::Venue`	drop	the two sealed venue witnesses; each module constructs only its own
`PackageWitness`	drop	pinned but never constructed (no pause bits)

2. Value flows

SUI

- Launch creation fee: `finalize` takes exactly `LaunchConfig.creation_fee` into the launch `Ledger.sui`; leaves by `claim_protocol_fees[_with_partner]` to the treasury (partner cut first).
- Bluefin pool creation fee: the caller of `bluefin::migrate` pays `creation_fee: Coin<SUI>` whole to Bluefin.
- Nothing else touches SUI; no path assumes a gas coin.

Quote

- Launch: `buy` → `real_quote_reserve` (net of the Quote fee, which splits into the launch `Ledger` and `creator_quote_fees`); refund of input above the graduation cap returns to the buyer. `sell` ← `real_quote_reserve` (gross output, fee split as above). Opening round: `join` → `opening_round_quote`; `settle` moves the pot through `buy_from_curve` and keeps the refund for pro-rata delivery. `migrate`: `real_quote_reserve` (== target) → recipients (paid or vested) + `Ledger` (Blast's share, rounded up) + `Migration.quote` (pool share).
- Presale: `commit_*` → `quote` (hard cap returns the excess coin at once); `migrate` splits `raise_target` into Blast's `Ledger`, raise recipients, and `Migration.quote`; the overflow above the target stays for pro-rata refunds in `claim`; `cancel` keeps everything for full refunds.
- Vault: residual Quote from pool seeding → `Ledger` (protocol policy; paid at once unless a partner is bound) or `owner_quote`; LP-fee collections → Blast's cut (rounded up) into the `Ledger`, the rest apportioned to address routes (paid), `owner_quote`, `auto_lp_quote`, `rewards_quote`, `burn_quote`; `rewards_quote` → `Round.quote` + optional gas reserve; gas reserve → frozen treasury (≤ reserve) and back to the vault; `burn_quote` → venue swap → Meme burned, unspent back.

Meme

- Minted once into the `LaunchTicket` (launchpad) or by `finalize` (presale); supply then becomes `Fixed` or `BurnOnly` and no `TreasuryCap` survives.
- Launch: curve share → `meme_reserve`; the rest → `allocated_meme`; sells put `pending_burn` aside; `migrate` pays recipients, accrues Blast's share into the `Ledger`, burns (or sends to `0x0`) `meme_reserve + pending_burn`, hands the pool share off.
- Presale: `sale_tokens` → participants (paid or vested) pro rata on claim; `liquidity_tokens` → `Migration.meme`; `recipient_tokens` → recipients at migration; `protocol_tokens` → `Ledger`. After cancellation every token balance stays in the presale for good.
- Vault: residual Meme and LP-fee Meme as for Quote; the burn route's Meme is burned (`BurnOnly`) or sent to `0x0` (`Fixed`); `rewards_meme` → `Round.meme`.

3. Invariants

blast_admin

- One `Publication<OTW>` per package, consumed exactly once by `wrap`, which accepts only the original package's never-upgraded cap.
- `PackageVersion.version` moves only by `+1` inside `commit_upgrade_and_advance`; every consumer compares its compiled constant with `assert_current`.
- Every pause bit set is in `valid_pause_bits`; clearing requires the pinned package witness and a guarded upgrade delay.
- A delayed proposal becomes effective only after `ready_at_ms` computed from the **applied** delay at scheduling; no delay exceeds 30 days; a replacement cancels and restarts; destruction with a pending proposal aborts.
- A role check is `AdminWitness<Role>` minted by `sign_in` against the live registry; revocation is immediate.

blast_core

- `Ledger` pays only to the treasury destination, and with the bound partner first when one is bound; it is destroyed only empty.
- `apportion` returns amounts summing exactly to the total for rates summing to 10,000 (PoC: property over 300 random vectors).
- `Migration` and `OwnerProof` cannot be dropped or stored; `Migration` is redeemable only by the stored venue; terms carry the sealing witness, and `new` requires the same witness.
- `sqrt_price_x64` is the exact floor square root (PoC).
- Merkle leaves and nodes are domain-separated (`0x00` / `0x01`); proofs are positional with exact depth.

blast_vault

- `vault::new` requires the calling venue's witness and the terms' sealing witness both admitted; one vault per `(issuer, origin)` by derived ID; a vault leaves its creating transaction only through `share_with_position` with a positive-liquidity position.
- Each collection: `gross = protocol cut + Σ route parts`; Blast's cut rounds up; routes sum to 10,000 so nothing is left in the collection balances (`destroy_zero`).
- Venue receipts bind to the vault and cap what returns at what was taken.
- Rewards: `rewards_* ≥ Σ scheduled budgets`; a round's number is `next_round` and its interval starts at or after the last checkpoint; a leaf pays once (bitmap); unclaimed and unsettled balances return to the same vault at the deadline.
- Owner actions need an `OwnerProof` for the vault's exact terms; Blast's share and the fee split change only after a fixed 48 h.

blast_fun

- One launch per Meme; `Meme != Quote`; the admitted Quote, venue and decimals are checked at `new`; trade fees are snapshotted; the sell burn is ≤ 5,000 bps and the supply exactly `10^9 · 10^decimals` at `finalize`.
- Curve: `real_quote_reserve < graduation_target` while `Live`; a buy's net input never exceeds the gap (PoC: capped net == gap exactly); `meme_reserve > 0` always; buy-then-sell and split buys never profit (PoC); fees round against the trader; Blast's share of each fee rounds up.
- Opening round: `Σ undelivered bids == opening_round_bids`; delivery is pro rata, the last takes the remainder.
- `migrate` empties `meme_reserve`, `pending_burn`, `allocated_meme` and `real_quote_reserve`; recipients receive exactly their apportioned shares; Blast's one-time shares round up.
- Creator authority: one-time claim bound to sender (address) or to a signed `(domain, meme, commitment, claimant, expiry)` message; the launch honours one cap ID per authority; reassignment waits 48 h.

blast_presale

- One presale per coin; the sale token's `Currency` is `Unregulated` with `Unknown` supply and a zero-supply cap at `new`; `finalize` mints the whole supply once and seals it.
- Token shares sum to 10,000 with Blast's rate; raise shares plus Blast's leave a positive LP share; the LP seed price is seedable at creation and after every rate change.
- `Σ commitments == total_commitments == quote.value()` until migration or claims; under a hard cap `total ≤ raise_target`.
- Claims: tokens pro rata to `token_allocation` when migrated, Quote pro rata to the overflow (migrated) or everything (cancelled); the last claim takes the remainder; `participants` counts undelivered commitments.
- `migrate` moves exactly `raise_target` out: Blast's Quote, raise recipients, LP share; the LP tokens and all recipient tokens leave; sale tokens stay for claims.

blast_migrators

- A venue redeems only its own `Migration`; the pool pair must be exactly the sale's Meme and Quote (`is_meme_coin_a`), in Cetus's canonical order for Cetus.
- Cetus: pool creation is reserved at `prepare` with the Meme's single `PoolCreationCap` at the recorded tick spacing; migration creates the pool, seeds full range from the exact hand-off balances, burns the position and vaults the burn proof and residuals.
- Bluefin: migration creates a fresh `Pool<Meme, Quote>` at the recorded tier and seeds full range; the position is vaulted.
- Collections touch only the vault's own pool (`pool_id` check) and route everything through `vault::split_fees`; auto-LP adds only what the venue's own math says the other side covers.

4. State tables

Launch (`launch.move`)

Rows are functions that read the lifecycle; cells are `ok` or the abort. `V` = version gate, `P<bit>` = pause bit. Every row also requires `V`.

Function	OpeningRound	Live	MigrationReady	Migrated
`join_opening_round` (P2, before end)	ok	invalid_launch_lifecycle (12)	12	12
`settle_opening_round` (P2, after end)	ok → Live (→ MigrationReady if the pot reaches the target)	12	12	12
`deliver_opening_round`	12	ok	ok	ok
`buy` (P2)	12	ok (→ MigrationReady at the target)	12	12
`sell` (P2, shared)	12	ok	12	12
`migrate` (P4)	12	12	ok → Migrated	12
`claim_creator_caps` (shared)	ok	ok	ok	ok
`claim_creator_fees` / `claim_protocol_fees[_with_partner]`	ok	ok	ok	ok
cap reassignment schedule/apply/cancel, metadata setters, `prove_owner`	ok	ok	ok	ok
`share`	ok once (unshared only)	ok once	ok once	ok once

Terminal: `Migrated`. Stuck cells: `MigrationReady` leaves only through the stored venue (accepted LP-I6); `OpeningRound` settles only while trading is unpaused (accepted LP-I3).

Presale (`blast_presale.move`)

Function	Pending	Open	Funded	Migrated	Cancelled
`share` (P1)	ok → Open	11	11	11	11
`set_protocol_fees` (FeeAdmin)	ok (≤ policy)	ok before `start_ms`, rates may only fall	11	11	11
`commit_open` / `commit_allowlisted` / `commit_uncapped` (P2, `start ≤ now < end`)	11	ok	11	11	11
`settle` (after end, or early when full under a closing hard cap)	11	ok → Funded or Cancelled	11	11	11
`cancel_unlaunched` (≥ LP deadline)	11	11	ok → Cancelled	11	11
`migrate` (P4)	11	11	ok → Migrated	11	11
`claim`	11	11	11	ok	ok
`pay_protocol_fees_with_partner`	no-op	no-op	no-op	ok	no-op
`prove_owner`	ok	ok	ok	ok	ok
`destroy_cap`	11	11	11	11	ok

Terminal: `Migrated` (tokens claimable), `Cancelled` (Quote refundable, tokens locked for good). No cell leaves funds without an exit: `Funded` reaches `Cancelled` permissionlessly at the LP deadline.

Vault (`vault.move`, `holder_rewards.move`)

The vault has one state; its slot machine is the round publication:

Slot state	`schedule_round` (RewardsAdmin)	`publish_round` (anyone)	`cancel_round` (CancelAdmin)	`schedule_round_delay`	`apply_round_delay`	`cancel_round_delay`
empty	ok → round pending	no_pending (14)	14	ok → delay pending	14	14
round pending	replaces (restarts)	not_ready (13) until ready, then ok → empty, shares `Round`	ok → empty	replaces	wrong_change (18)	wrong_round_proposal (20)
delay pending	replaces	wrong_change (18)	wrong_round_proposal (20)	replaces	not_ready until ready, then ok	ok → empty

`Round`: open until `claim_deadline_at_ms`; `claim` needs `now < deadline`; `settle_gas_reserve` once while the round exists; `close_round` needs `now ≥ deadline` and deletes it.

Pause (`admin_pause_bit`) blocks: `schedule_protocol_bps`, `apply_protocol_bps`, `schedule_round`, `publish_round`, `schedule_round_delay`, `apply_round_delay`, `settle_gas_reserve`, `take_buyback_quote`. Never paused: collection, owner claims and split changes, reward claims, `close_round`, cancels, venue admission.

Delayed primitives (`delayed.move`, `delayed_action.move`, `delayed_set.move`, `delayed_map.move`)

	`schedule*`	`apply` / `take_action` / `apply_delay`	`cancel`	`destroy`
nothing pending	ok, `ready_at = now + applied`	no_pending (14)	14	ok
pending, not ready	replaces (emits Cancelled then Scheduled)	not_ready (13)	ok	proposal_pending (17)
pending, ready	replaces	ok, commits value and next delay	ok	17

Kind checks: `take_action` on a delay change and `apply_delay` on an action abort `wrong_change` (18). A proposed delay above 30 days aborts `invalid_delay` (15) at scheduling.

PackageVersion and PackageAdmin

	`paused` bits	`pause` (PauseAdmin)	`package unpause` (UnpauseAdmin + witness)	`schedule_upgrade`	`authorize_upgrade` → `commit_upgrade_and_advance`	`make_immutable`
fresh wrap	all defined bits	pause_unchanged if nothing new	needs every covered delay guarded and the upgrade delay guarded	ok (48 h)	after the notice, version `+1`	after a scheduled immutability action

The products' `unpause` wrappers check `assert_current`; the vault's does not (finding L-1).

Appendix B. Function inventory

Total: 593 functions (blast_admin 175, blast_core 79, blast_vault 94, blast_fun 143, blast_presale 76, blast_migrators 26). Every `public`, `public(package)` and `entry` function of the six in-scope packages at `4fe8d739f04bafd0775691795eb9f3727e984089`, excluding `#[test_only]` functions. Columns are derived mechanically from each body and signature (`tools/function-inventory-raw.tsv` in the evidence directory holds the raw extraction) and were checked by hand during the review:

- **Version**: the body calls `assert_current` on a `PackageVersion`. `blast_admin`'s generic calls are deliberately ungated (ADR 0137); `blast_core` is immutable and, apart from the treasury, stateless.
- **Pause**: the body calls `assert_not_paused`.

- **Auth**: the role witness the signature takes (`AdminWitness<Role>`), the root cap, a creator cap, an owner proof, a sealed witness value, an owned object the caller must hold, or none (permissionless, or acting on a caller-owned value).
- **Value**: whether coins or balances enter (`in`) or a coin, balance, hand-off, ticket, upgrade ticket or proof leaves (`out`).
- **Events**: emitters called in the body. Delayed-primitive lifecycle events are emitted by the primitive and are not repeated.
- **Rounding**: `up` when the body uses `apply_up`, `div_ceil` or `mul_div_ceil`; `down` when it uses `mul_div` or `/`.

State preconditions and the state tables are in system-map.md; the guard order of each economic function was read in execution order and is cited in the report where it matters.

blast_admin (175 functions)

`blast_admin::blast_acl`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`grant_admin`	public	—	—	root cap	—	admin_granted	—
`revoke_admin`	public	—	—	root cap	—	admin_revoked	—
`grant_role`	public	—	—	root cap	—	admin_role_granted	—
`revoke_role`	public	—	—	root cap	—	admin_role_revoked	—
`destroy_admin_cap`	public	—	—	none	—	admin_destroyed	—
`begin_super_admin_transfer`	public	—	—	root cap	—	—	—
`cancel_super_admin_transfer`	public	—	—	root cap	—	—	—
`accept_super_admin_transfer`	public	—	—	none	—	super_admin_transferred	—
`destroy_retired_super_admin_cap`	public	—	—	root cap	—	super_admin_cap_destroyed	—
`sign_in`	public	—	—	none	—	—	—
`super_admin_cap_id`	public	—	—	none	—	—	—
`is_admin`	public	—	—	none	—	—	—
`has_role`	public	—	—	none	—	—	—
`admin_roles`	public	—	—	none	—	—	—
`admin_count`	public	—	—	none	—	—	—
`super_admin_transfer`	public	—	—	none	—	—	—
`max_roles_per_admin`	public	—	—	none	—	—	—
`super_admin_transfer_delay_ms`	public	—	—	none	—	—	—
`witness_for_testing`	public	—	—	none	—	—	—

`blast_admin::blast_admin_events`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`paused`	package	—	—	none	—	Paused	—
`unpaused`	package	—	—	none	—	Unpaused	—
`admin_granted`	package	—	—	none	—	AdminGranted	—
`admin_revoked`	package	—	—	none	—	AdminRevoked	—
`admin_role_granted`	package	—	—	none	—	AdminRoleGranted	—
`admin_role_revoked`	package	—	—	none	—	AdminRoleRevoked	—
`admin_destroyed`	package	—	—	none	—	AdminDestroyed	—
`super_admin_transferred`	package	—	—	none	—	SuperAdminTransferred	—
`super_admin_cap_destroyed`	package	—	—	none	—	SuperAdminCapDestroyed	—
`version_advanced`	package	—	—	none	—	VersionAdvanced	—
`package_version_created`	package	—	—	none	—	PackageVersionCreated	—
`package_admin_created`	package	—	—	none	—	PackageAdminCreated	—
`upgrade_authorized`	package	—	—	none	—	UpgradeAuthorized	—
`upgrade_committed`	package	—	—	none	—	UpgradeCommitted	—
`upgrade_cap_destroyed`	package	—	—	none	—	UpgradeCapDestroyed	—

`blast_admin::delayed`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`new`	public	—	—	none	—	DelayedCreated	—
`max_delay_ms`	public	—	—	none	—	—	—
`value`	public	—	—	none	—	—	—
`delay_ms`	public	—	—	none	—	—	—
`is_guarded`	public	—	—	none	—	—	—

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`pending`	public	—	—	none	—	—	—
`schedule`	public	—	—	none	—	DelayedScheduled	—
`schedule_value`	public	—	—	none	—	—	—
`schedule_delay`	public	—	—	none	—	—	—
`assert_ready`	public	—	—	none	—	—	—
`apply`	public	—	—	none	—	DelayedApplied	—
`cancel`	public	—	—	none	—	—	—
`destroy`	public	—	—	none	—	DelayedDestroyed	—
`proposal_value`	public	—	—	none	—	—	—
`proposal_delay_ms`	public	—	—	none	—	—	—
`proposal_ready_at_ms`	public	—	—	none	—	—	—
`pending_delay_ms`	public	—	—	none	—	—	—
`pending_delay_ready_at_ms`	public	—	—	none	—	—	—
`owner`	public	—	—	none	—	—	—
`key`	public	—	—	none	—	—	—
`is_ready`	public	—	—	none	—	—	—
`new_proposal`	package	—	—	none	—	—	—
`new_pending_delay`	package	—	—	none	—	—	—
`assert_delay`	package	—	—	none	—	—	—
`ready_at_ms_after`	package	—	—	none	—	—	—
`is_ready_at`	package	—	—	none	—	—	—
`assert_ready_at`	package	—	—	none	—	—	—

`blast_admin::delayed_action`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`new`	public	—	—	none	—	ActionCreated	—
`delay_ms`	public	—	—	none	—	—	—
`is_guarded`	public	—	—	none	—	—	—
`pending`	public	—	—	none	—	—	—
`schedule`	public	—	—	none	—	—	—
`schedule_delay`	public	—	—	none	—	—	—
`take_action`	public	—	—	none	—	—	—
`apply_delay`	public	—	—	none	—	—	—
`cancel`	public	—	—	none	—	—	—
`destroy`	public	—	—	none	—	ActionDestroyed	—
`action`	public	—	—	none	—	—	—
`pending_action`	public	—	—	none	—	—	—
`pending_delay`	public	—	—	none	—	—	—
`is_ready`	public	—	—	none	—	—	—
`is_delay_ready`	public	—	—	none	—	—	—
`owner`	public	—	—	none	—	—	—
`key`	public	—	—	none	—	—	—

`blast_admin::delayed_map`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`new`	public	—	—	none	—	MapCreated	—
`value`	public	—	—	none	—	—	—
`delay_ms`	public	—	—	none	—	—	—
`pending`	public	—	—	none	—	—	—
`pending_delay`	public	—	—	none	—	—	—
`schedule`	public	—	—	none	—	MapScheduled	—
`apply`	public	—	—	none	—	MapApplied	—
`cancel`	public	—	—	none	—	—	—
`schedule_delay`	public	—	—	none	—	MapScheduled	—
`apply_delay`	public	—	—	none	—	MapApplied	—

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`cancel_delay`	public	—	—	none	—	—	—
`destroy`	public	—	—	none	—	MapDestroyed	—
`entry_proposal_value`	public	—	—	none	—	—	—
`entry_proposal_delay`	public	—	—	none	—	—	—
`is_pending`	public	—	—	none	—	—	—
`entry_ready_at_ms`	public	—	—	none	—	—	—
`is_ready`	public	—	—	none	—	—	—
`is_delay_ready`	public	—	—	none	—	—	—
`owner`	public	—	—	none	—	—	—
`key`	public	—	—	none	—	—	—
`contains`	public	—	—	none	—	—	—
`length`	public	—	—	none	—	—	—
`pending_count`	public	—	—	none	—	—	—
`is_guarded`	public	—	—	none	—	—	—
`changed_entry`	public	—	—	none	—	—	—
`changed_value`	public	—	—	none	—	—	—
`cancel_entry_if_pending`	entry	—	—	none	—	MapCancelled	—

`blast_admin::delayed_set`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`new`	public	—	—	none	—	SetCreated	—
`contains`	public	—	—	none	—	—	—
`members`	public	—	—	none	—	—	—
`delay_ms`	public	—	—	none	—	—	—
`admission_ready_at_ms`	public	—	—	none	—	—	—
`pending_count`	public	—	—	none	—	—	—
`is_guarded`	public	—	—	none	—	—	—
`pending_delay`	public	—	—	none	—	—	—
`schedule_add`	public	—	—	none	—	SetScheduled	—
`apply_add`	public	—	—	none	—	SetApplied	—
`cancel_add`	public	—	—	none	—	—	—
`remove`	public	—	—	none	—	MemberRemoved	—
`schedule_delay`	public	—	—	none	—	SetScheduled	—
`apply_delay`	public	—	—	none	—	SetApplied	—
`cancel_delay`	public	—	—	none	—	—	—
`destroy`	public	—	—	none	—	SetDestroyed	—
`length`	public	—	—	none	—	—	—
`is_pending`	public	—	—	none	—	—	—
`is_ready`	public	—	—	none	—	—	—
`is_delay_ready`	public	—	—	none	—	—	—
`owner`	public	—	—	none	—	—	—
`key`	public	—	—	none	—	—	—
`admission_delay`	public	—	—	none	—	—	—
`member`	public	—	—	none	—	—	—

`blast_admin::package_admin`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`wrap`	public	—	—	owned object	—	—	—
`schedule_upgrade`	public	—	—	UpgradeAdmin<OTW	—	—	—
`schedule_make_immutable`	public	—	—	UpgradeAdmin<OTW	—	—	—
`cancel_upgrade_action`	public	—	—	CancelAdmin<OTW	—	—	—
`authorize_upgrade`	public	—	—	UpgradeAdmin<OTW	out	—	—
`commit_upgrade_and_advance`	public	—	—	UpgradeAdmin<OTW	—	—	—
`make_immutable`	public	—	—	UpgradeAdmin<OTW, sealed witness	—	—	—
`schedule_upgrade_delay`	public	—	—	UpgradeAdmin<OTW	—	—	—

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
apply_upgrade_delay	public	—	—	none	—	—	—
pause	public	—	—	PauseAdmin<OTW	—	—	—
clear_pause_bits	public	—	—	UnpauseAdmin<OTW, sealed witness	—	—	—
publisher	public	—	—	sealed witness	—	—	—
publisher_mut	public	—	—	sealed witness	—	—	—
upgrade_delay_ms	public	—	—	none	—	—	—
pending_upgrade	public	—	—	none	—	—	—
is_upgrade_delay_guarded	public	—	—	none	—	—	—
has_upgrade_cap	public	—	—	none	—	—	—
upgrade_cap_id	public	—	—	none	—	—	—
current_package_id	public	—	—	none	—	—	—
upgrade_cap_version	public	—	—	none	—	—	—
upgrade_policy	public	—	—	none	—	—	—
publisher_id	public	—	—	none	—	—	—
upgrade_digest	public	—	—	none	—	—	—
initial_upgrade_delay_ms	public	—	—	none	—	—	—
new_for_testing	public	—	—	owned object	—	—	—
authorize_upgrade_for_testing	public	—	—	UpgradeAdmin<OTW	out	—	—

blast_admin::package_version

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
new_publication	public	—	—	sealed witness	—	—	—
keep	public	—	—	owned object	—	—	—
assert_current	public	—	—	none	—	—	—
assert_not_paused	public	—	—	none	—	—	—
version	public	—	—	none	—	—	—
valid_pause_bits	public	—	—	none	—	—	—
paused	public	—	—	none	—	—	—
is_paused	public	—	—	none	—	—	—
package_witness	public	—	—	none	—	—	—
publication_valid_pause_bits	public	—	—	owned object	—	—	—
publication_publisher_id	public	—	—	owned object	—	—	—
publication_package_witness	public	—	—	owned object	—	—	—
into_version	package	—	—	owned object	—	—	—
share	package	—	—	none	—	—	—
assert_unpause_witness	package	—	—	none	—	—	—
assert_package_witness	package	—	—	none	—	—	—
set_pause_bits	package	—	—	none	—	—	—
clear_pause_bits	package	—	—	none	—	—	—
advance	package	—	—	none	—	—	—
new_for_testing	public	—	—	none	—	—	—

blast_core (79 functions)

blast_core::blast_core_events

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
protocol_fees_paid	package	—	—	none	—	ProtocolFeesPaid	—

blast_core::blast_treasury

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
schedule_destination	public	—	—	TreasuryAdmin	—	—	—
apply_destination	public	—	—	none	—	—	—
cancel_destination	public	—	—	CancelAdmin	—	—	—
destination	public	—	—	none	—	—	—
delay_ms	public	—	—	none	—	—	—

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`pending_destination`	public	—	—	none	—	—	—
`is_destination_ready`	public	—	—	none	—	—	—
`is_guarded`	public	—	—	none	—	—	—

`blast_core::bps`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`new`	public	—	—	none	—	—	—
`value`	public	—	—	none	—	—	—
`apply_up`	public	—	—	none	—	—	up
`apportion`	public	—	—	none	—	—	up/down
`sum`	package	—	—	none	—	—	—

`blast_core::fee_split`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`address_destination`	public	—	—	none	—	—	—
`owner_destination`	public	—	—	none	—	—	—
`auto_lp_destination`	public	—	—	none	—	—	—
`holder_rewards_destination`	public	—	—	none	—	—	—
`burn_destination`	public	—	—	none	—	—	—
`route`	public	—	—	none	—	—	—
`new`	public	—	—	none	—	—	—
`apportion`	public	—	—	none	—	—	—
`routes`	public	—	—	none	—	—	—
`destination`	public	—	—	none	—	—	—
`bps`	public	—	—	none	—	—	—
`address_of`	public	—	—	none	—	—	—
`is_owner`	public	—	—	none	—	—	—
`is_auto_lp`	public	—	—	none	—	—	—
`is_holder_rewards`	public	—	—	none	—	—	—
`is_burn`	public	—	—	none	—	—	—

`blast_core::merkle`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`leaf`	public	—	—	none	—	—	—
`node`	public	—	—	none	—	—	—
`verify`	public	—	—	none	—	—	down

`blast_core::migration`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`residuals_to_protocol`	public	—	—	none	—	—	—
`residuals_to_owner`	public	—	—	none	—	—	—
`issue_ticket`	public	—	—	sealed witness	out	—	—
`redeem_ticket`	public	—	—	none	—	—	—
`ticket_venue`	public	—	—	none	—	—	—
`vault_terms`	public	—	—	sealed witness	—	—	—
`new`	public	—	—	sealed witness	in/out	—	—
`redeem`	public	—	—	sealed witness	out	—	—
`venue`	public	—	—	none	—	—	—
`balances`	public	—	—	none	—	—	—
`terms`	public	—	—	none	—	—	—
`origin`	public	—	—	none	—	—	—
`package_witness`	public	—	—	none	—	—	—
`fee_split`	public	—	—	none	—	—	—
`protocol_bps`	public	—	—	none	—	—	—

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`partner_id`	public	—	—	none	—	—	—
`residuals`	public	—	—	none	—	—	—
`is_protocol`	public	—	—	none	—	—	—
`is_owner`	public	—	—	none	—	—	—
`sqrt_price_x64`	public	—	—	none	—	—	down
`is_seedable_price`	public	—	—	none	—	—	down

`blast_core::owner_proof`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`new`	public	—	—	sealed witness	out	—	—
`assert_terms`	public	—	—	owner proof	—	—	—
`package_witness`	public	—	—	owner proof	—	—	—
`origin`	public	—	—	owner proof	—	—	—

`blast_core::payout`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`vesting_terms`	public	—	—	none	—	—	down
`recipient`	public	—	—	none	—	—	—
`destination`	public	—	—	none	—	—	—
`recipient_vesting_terms`	public	—	—	none	—	—	—
`cliff_delay_ms`	public	—	—	none	—	—	—
`period_duration_ms`	public	—	—	none	—	—	—
`periods`	public	—	—	none	—	—	—
`pay_or_vest`	public	—	—	none	in	—	—
`pay`	public	—	—	none	in	—	—
`pay_all`	public	—	—	none	in	—	—

`blast_core::protocol_fees`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`new`	public	—	—	sealed witness	—	—	—
`accrue`	public	—	—	none	in	—	—
`accrue_sui`	public	—	—	none	in	—	—
`take_cut`	public	—	—	none	in	—	up
`pay`	public	—	—	none	—	—	—
`pay_with_partner`	public	—	—	none	—	—	—
`destroy`	public	—	—	none	—	—	—
`balances`	public	—	—	none	—	—	—
`package_witness`	public	—	—	none	—	—	—
`origin`	public	—	—	none	—	—	—
`partner_id`	public	—	—	none	—	—	—

blast_vault (94 functions)

`blast_vault::blast_vault_events`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`vault_created`	package	—	—	none	—	VaultCreated	—
`fees_collected`	package	—	—	none	—	FeesCollected	—
`auto_lp_position_opened`	package	—	—	none	—	AutoLpPositionOpened	—
`auto_liquidity_added`	package	—	—	none	—	AutoLiquidityAdded	—
`buyback_burned`	package	—	—	none	—	BuybackBurned	—
`owner_balances_claimed`	package	—	—	none	—	OwnerBalancesClaimed	—
`round_published`	package	—	—	none	—	RoundPublished	—
`reward_claimed`	package	—	—	none	—	RewardClaimed	—
`gas_reserve_settled`	package	—	—	none	—	GasReserveSettled	—
`round_closed`	package	—	—	none	—	RoundClosed	—

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`auto_liquidity_added_for_testing`	public	—	—	none	—	—	—
`buyback_burned_for_testing`	public	—	—	none	—	—	—

`blast_vault::blast_vault_package`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`schedule_witness`	public	yes	—	VenueAdmin	—	—	—
`apply_witness`	public	yes	—	none	—	—	—
`cancel_witness`	public	yes	—	CancelAdmin	—	—	—
`remove_witness`	public	yes	—	VenueAdmin	—	—	—
`schedule_witness_delay`	public	yes	—	VenueAdmin	—	—	—
`apply_witness_delay`	public	yes	—	none	—	—	—
`cancel_witness_delay`	public	yes	—	CancelAdmin	—	—	—
`unpause`	public	—	—	UnpauseAdmin<BLAST_VAULT_PACKAGE	—	—	—
`make_immutable`	public	yes	—	UpgradeAdmin<BLAST_VAULT_PACKAGE	—	—	—
`assert_venue`	package	—	—	none	—	—	—
`assert_package_witness`	package	—	—	none	—	—	—
`uid_mut`	package	—	—	none	—	—	—
`registry_for_testing`	public	—	—	none	—	—	—
`empty_registry_for_testing`	public	—	—	none	—	—	—
`version_for_testing`	public	—	—	none	—	—	—

`blast_vault::holder_rewards`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`schedule_round`	public	yes	yes	RewardsAdmin	—	—	—
`publish_round`	public	yes	yes	none	—	—	up
`cancel_round`	public	yes	—	CancelAdmin	—	—	—
`schedule_round_delay`	public	yes	yes	RewardsAdmin	—	—	—
`apply_round_delay`	public	yes	yes	none	—	—	—
`cancel_round_delay`	public	yes	—	CancelAdmin	—	—	—
`claim`	public	yes	—	none	—	—	—
`settle_gas_reserve`	public	yes	yes	RewardsAdmin	—	—	—
`close_round`	public	yes	—	none	—	—	—
`reward_leaf_for_testing`	public	—	—	none	—	—	—
`round_balances_for_testing`	public	—	—	none	—	—	—
`is_claimed_for_testing`	public	—	—	none	—	—	—

`blast_vault::vault`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`new`	public	yes	—	sealed witness	—	—	—
`share_with_position`	public	—	—	sealed witness	in	—	—
`position_mut`	public	yes	—	sealed witness	—	—	—
`has_auto_lp_position`	public	—	—	sealed witness	—	—	—
`attach_auto_lp_position`	public	yes	—	sealed witness	—	—	—
`auto_lp_position`	public	—	—	sealed witness	—	—	—
`auto_lp_position_mut`	public	yes	—	sealed witness	—	—	—
`split_fees`	public	yes	—	sealed witness	in	—	—
`assert_fee_split_ready`	public	—	—	none	—	—	—
`apply_fee_split`	public	yes	—	sealed witness	—	—	—
`take_auto_lp`	public	yes	—	sealed witness	out	—	—
`keep_auto_lp`	public	yes	—	sealed witness	in	—	—
`take_buyback_quote`	public	yes	yes	BuybackAdmin, sealed witness	out	—	—
`burn_buyback`	public	yes	—	sealed witness	in	—	—
`claim_owner_balances`	public	yes	—	owner proof	out	—	—
`schedule_fee_split`	public	yes	—	owner proof	—	—	—
`cancel_fee_split`	public	yes	—	owner proof	—	—	—

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
schedule_protocol_bps	public	yes	yes	FeeAdmin	—	—	—
apply_protocol_bps	public	yes	yes	none	—	—	—
cancel_protocol_bps	public	yes	—	CancelAdmin	—	—	—
pay_protocol_fees_with_partner	public	yes	—	none	—	—	—
pool_id	public	—	—	none	—	—	—
new_round_terms	package	—	—	none	—	—	—
schedule_round_publication	package	—	—	none	—	—	—
take_round_publication	package	—	—	none	—	—	—
cancel_round_publication	package	—	—	none	—	—	—
schedule_round_publication_delay	package	—	—	none	—	—	—
apply_round_publication_delay	package	—	—	none	—	—	—
cancel_round_publication_delay	package	—	—	none	—	—	—
fund_round	package	—	—	none	out	—	—
restore_rewards	package	—	—	none	in	—	—
rewards_balances	package	—	—	none	—	—	—
next_round	package	—	—	none	—	—	—
last_checkpoint	package	—	—	none	—	—	—
number	package	—	—	none	—	—	—
start_checkpoint	package	—	—	none	—	—	—
end_checkpoint	package	—	—	none	—	—	—
root	package	—	—	none	—	—	—
manifest_digest	package	—	—	none	—	—	—
leaf_count	package	—	—	none	—	—	—
meme_amount	package	—	—	none	—	—	—
quote_amount	package	—	—	none	—	—	—
gas_reserve_amount	package	—	—	none	—	—	—
claim_window_duration_ms	package	—	—	none	—	—	—
expected_treasury	package	—	—	none	—	—	—
vault_key_for_testing	public	—	—	none	—	—	—
balances_for_testing	public	—	—	none	—	—	—
protocol_balances_for_testing	public	—	—	none	—	—	—
fee_split_for_testing	public	—	—	none	—	—	—
protocol_bps_for_testing	public	—	—	none	—	—	—
round_publication_for_testing	public	—	—	none	—	—	—
fund_rewards_for_testing	public	—	—	none	in	—	—
withdraw_rewards_for_testing	public	—	—	none	out	—	—
fund_burn_quote_for_testing	public	—	—	none	in	—	—
share_for_testing	public	—	—	none	—	—	—

blast_fun (143 functions)

blast_fun::allocation

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
recipient_share	public	—	—	none	—	—	—
plan	public	—	—	none	—	—	—
meme_bps	package	—	—	none	—	—	—
quote_bps	package	—	—	none	—	—	—
protocol_allocation	package	—	—	none	—	—	—
add_protocol	package	—	—	none	—	—	—
apportion_meme	package	—	—	none	—	—	—
apportion_quote	package	—	—	none	—	—	—
payees	package	—	—	none	—	—	—
protocol_allocation_unchecked_for_testing	public	—	—	none	—	—	—

blast_fun::blast_fun

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`package_witness`	package	—	—	none	—	—	—
`init_for_testing`	public	—	—	none	—	—	—
`version_for_testing`	public	—	—	none	—	—	—
`package_admin_for_testing`	public	—	—	none	—	—	—
`clear_pause_bits_for_testing`	public	—	—	none	—	—	—

`blast_fun::creator_authority`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`new_address`	public	—	—	none	—	—	—
`new_account`	public	—	—	none	—	—	—
`destroy_curve_fee_cap`	public	—	—	none	—	—	—
`destroy_creator_metadata_cap`	public	—	—	none	—	—	—
`claim_creator_caps`	package	—	—	none	—	—	—
`new_curve_fee_cap`	package	—	—	none	—	—	—
`new_creator_metadata_cap`	package	—	—	none	—	—	—

`blast_fun::curve`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`cap_buy_quote_input`	package	—	—	none	—	—	up
`quote_buy`	package	—	—	none	—	—	up
`quote_sell`	package	—	—	none	—	—	up

`blast_fun::launch`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`new`	public	yes	yes	owned object	—	—	—
`set_virtual_quote`	public	yes	—	hot potato holder	—	—	—
`set_graduation_target`	public	yes	—	hot potato holder	—	—	—
`set_allocation_plan`	public	yes	—	hot potato holder	—	—	—
`set_lp_fee_split`	public	yes	—	hot potato holder	—	—	—
`set_opening_round`	public	yes	—	hot potato holder	—	—	—
`set_partner`	public	yes	—	hot potato holder	—	—	—
`finalize`	public	yes	yes	hot potato holder	in	—	—
`share`	public	yes	yes	none	—	—	—
`claim_creator_caps`	public	yes	—	none	—	—	—
`schedule_curve_fee_cap_delay`	public	yes	—	CreatorAdmin	—	—	—
`schedule_curve_fee_cap_reassignment`	public	yes	—	CreatorAdmin	—	—	—
`cancel_curve_fee_cap_reassignment`	public	yes	—	CancelAdmin	—	—	—
`apply_curve_fee_cap_delay`	public	yes	—	none	—	—	—
`apply_curve_fee_cap_reassignment`	public	yes	—	none	—	—	—
`schedule_creator_metadata_cap_delay`	public	yes	—	CreatorAdmin	—	—	—
`schedule_creator_metadata_cap_reassignment`	public	yes	—	CreatorAdmin	—	—	—
`cancel_creator_metadata_cap_reassignment`	public	yes	—	CancelAdmin	—	—	—
`apply_creator_metadata_cap_delay`	public	yes	—	none	—	—	—
`apply_creator_metadata_cap_reassignment`	public	yes	—	none	—	—	—
`set_name`	public	—	—	creator cap	—	—	—
`set_description`	public	—	—	creator cap	—	—	—
`set_icon_url`	public	—	—	creator cap	—	—	—
`join_opening_round`	public	yes	yes	none	in	—	—
`settle_opening_round`	public	yes	yes	none	—	—	—
`deliver_opening_round`	public	yes	—	none	—	—	down
`buy`	public	yes	yes	none	in/out	—	—
`sell`	public	yes	yes	none	in/out	—	—
`claim_creator_fees`	public	yes	—	creator cap	out	—	—
`claim_protocol_fees`	public	yes	—	none	—	—	—
`claim_protocol_fees_with_partner`	public	yes	—	none	—	—	—

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
migrate	public	yes	yes	none	out	—	—
prove_owner	public	yes	—	creator cap	out	—	—
registry_for_testing	public	—	—	none	—	—	—
launch_key_for_testing	public	—	—	none	—	—	—
balances_for_testing	public	—	—	none	—	—	—
opening_round_bid_for_testing	public	—	—	none	—	—	—
opening_round_for_testing	public	—	—	none	—	—	—
fee_totals_for_testing	public	—	—	none	—	—	—

blast_fun::launch_config

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
schedule_protocol_allocation	public	yes	—	TreasuryAdmin	—	—	—
apply_protocol_allocation	public	yes	—	none	—	—	—
cancel_protocol_allocation	public	yes	—	CancelAdmin	—	—	—
schedule_protocol_lp_fees	public	yes	—	FeeAdmin	—	—	—
apply_protocol_lp_fees	public	yes	—	none	—	—	—
cancel_protocol_lp_fees	public	yes	—	CancelAdmin	—	—	—
set_meme_decimals_allowed	public	yes	—	ListingAdmin	—	meme_decimals_allowed	—
set_quote_allowed	public	yes	—	ListingAdmin	—	—	—
schedule_trade_fees	public	yes	—	FeeAdmin	—	—	—
apply_trade_fees	public	yes	—	none	—	—	—
cancel_trade_fees	public	yes	—	CancelAdmin	—	—	—
schedule_creation_fee	public	yes	—	FeeAdmin	—	—	—
apply_creation_fee	public	yes	—	none	—	—	—
cancel_creation_fee	public	yes	—	CancelAdmin	—	—	—
add_partner	public	yes	—	FeeAdmin	—	—	—
remove_partner	public	yes	—	FeeAdmin	—	—	—
schedule_venue	public	yes	—	VenueAdmin	—	—	—
apply_venue	public	yes	—	none	—	—	—
cancel_venue	public	yes	—	CancelAdmin	—	—	—
remove_venue	public	yes	—	VenueAdmin	—	—	—
schedule_venue_delay	public	yes	—	VenueAdmin	—	—	—
apply_venue_delay	public	yes	—	none	—	—	—
cancel_venue_delay	public	yes	—	CancelAdmin	—	—	—
unpause	public	yes	—	UnpauseAdmin<BLAST_FUN	—	—	—
snapshot_for_launch	package	—	—	none	—	—	—
protocol_allocation	package	—	—	none	—	—	—
protocol_lp_fees	package	—	—	none	—	—	—
is_guarded	package	—	—	none	—	—	—
creation_fee	package	—	—	none	—	—	—
assert_partner	package	—	—	none	—	—	—
venue_delay_ms_for_testing	public	—	—	none	—	—	—
set_protocol_allocation_for_testing	public	—	—	TreasuryAdmin	—	—	—
set_creation_fee_for_testing	public	—	—	FeeAdmin	—	—	—
set_trade_fees_for_testing	public	—	—	FeeAdmin	—	—	—
new_zero_fees_for_testing	public	—	—	none	—	—	—
init_zero_fees_for_testing	public	—	—	none	—	—	—
set_protocol_lp_fees_for_testing	public	—	—	FeeAdmin	—	—	—

blast_fun::launch_ticket

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
new	public	—	—	sealed witness	—	—	—
atomic_supply	package	—	—	none	—	—	—
id	package	—	—	owned object	—	—	—
decimals	package	—	—	owned object	—	—	—

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`destroy`	package	—	—	owned object	out	—	—
`set_sell_burn_bps_for_testing`	public	—	—	owned object	—	—	—
`burn_meme_for_testing`	public	—	—	owned object	—	—	—

`blast_fun::events`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`meme_decimals_allowed`	package	—	—	none	—	MemeDecimalsAllowed	—
`quote_allowed`	package	—	—	none	—	QuoteAllowed	—
`creator_caps_claimed`	package	—	—	none	—	CreatorCapsClaimed	—
`curve_fee_cap_reassigned`	package	—	—	none	—	CurveFeeCapReassigned	—
`creator_metadata_cap_reassigned`	package	—	—	none	—	CreatorMetadataCapReassigned	—
`meme_metadata_updated`	package	—	—	none	—	MemeMetadataUpdated	—
`launch_ticket_created`	package	—	—	none	—	LaunchTicketCreated	—
`launch_created`	package	—	—	none	—	LaunchCreated	—
`launch_shared`	package	—	—	none	—	LaunchShared	—
`opening_round_joined`	package	—	—	none	—	OpeningRoundJoined	—
`opening_round_settled`	package	—	—	none	—	OpeningRoundSettled	—
`opening_round_delivered`	package	—	—	none	—	OpeningRoundDelivered	—
`bought`	package	—	—	none	—	Bought	—
`sold`	package	—	—	none	—	Sold	—
`migration_allocation_paid`	package	—	—	none	—	MigrationAllocationPaid	—
`migrated`	package	—	—	none	—	Migrated	—
`creator_fees_claimed`	package	—	—	none	—	CreatorFeesClaimed	—
`protocol_fees_accrued`	package	—	—	none	—	ProtocolFeesAccrued	—

`blast_fun::runtime_config`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`schedule_creator_claim_signer`	public	yes	—	ClaimSignerAdmin	—	—	—
`apply_creator_claim_signer`	public	yes	—	none	—	—	—
`cancel_creator_claim_signer`	public	yes	—	CancelAdmin	—	—	—
`schedule_creator_claim_signer_delay`	public	yes	—	ClaimSignerAdmin	—	—	—
`apply_creator_claim_signer_delay`	public	yes	—	none	—	—	—
`cancel_creator_claim_signer_delay`	public	yes	—	CancelAdmin	—	—	—
`schedule_protocol_fee_share`	public	yes	—	FeeAdmin	—	—	—
`apply_protocol_fee_share`	public	yes	—	none	—	—	—
`cancel_protocol_fee_share`	public	yes	—	CancelAdmin	—	—	—
`revoke_creator_claim_signer`	public	yes	—	ClaimSignerAdmin	—	—	—
`is_guarded`	package	—	—	none	—	—	—
`creator_claim_signers`	package	—	—	none	—	—	—
`protocol_fee_share`	package	—	—	none	—	—	—
`set_protocol_fee_share_for_testing`	public	—	—	FeeAdmin	—	—	—
`new_publication_for_testing`	public	—	—	none	—	—	—
`new_zero_fee_share_for_testing`	public	—	—	none	—	—	—
`new_zero_fee_share_with_signer_for_testing`	public	—	—	none	—	—	—

blast_presale (76 functions)

`blast_presale::blast_presale`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`open_eligibility`	public	—	—	none	—	—	—
`allowlisted_eligibility`	public	—	—	none	—	—	—
`uncapped_for_eligible`	public	—	—	none	—	—	—
`uncapped_for_anyone`	public	—	—	none	—	—	—
`recipient_share`	public	—	—	none	—	—	—
`new`	public	yes	yes	owned object	—	—	—

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`set_supply`	public	yes	—	hot potato holder	—	—	—
`set_token_allocations`	public	yes	—	hot potato holder	—	—	—
`set_participant_vesting`	public	yes	—	hot potato holder	—	—	—
`set_raise`	public	yes	—	hot potato holder	—	—	—
`set_windows`	public	yes	—	hot potato holder	—	—	—
`set_eligibility`	public	yes	—	hot potato holder	—	—	—
`set_uncapped_phase`	public	yes	—	hot potato holder	—	—	—
`set_hard_cap`	public	yes	—	hot potato holder	—	—	—
`set_fee_split`	public	yes	—	hot potato holder	—	—	—
`set_partner`	public	yes	—	hot potato holder	—	—	—
`venue`	public	—	—	none	—	—	—
`finalize`	public	yes	—	hot potato holder	—	PresaleCreated	—
`share`	public	yes	yes	none	—	—	—
`set_protocol_fees`	public	yes	—	FeeAdmin	—	—	—
`commit_open`	public	yes	yes	none	in/out	—	—
`commit_allowlisted`	public	yes	yes	none	in/out	—	—
`commit_uncapped`	public	yes	yes	none	in/out	—	—
`settle`	public	yes	—	none	—	—	—
`cancel_unlaunched`	public	yes	—	none	—	—	—
`claim`	public	yes	—	none	—	—	—
`migrate`	public	yes	yes	none	out	—	—
`prove_owner`	public	yes	—	creator cap	out	—	—
`pay_protocol_fees_with_partner`	public	yes	—	none	—	—	—
`destroy_cap`	public	—	—	none	—	—	—
`registry_for_testing`	public	—	—	none	—	—	—
`presale_key_for_testing`	public	—	—	none	—	—	—
`open_eligibility_unchecked_for_testing`	public	—	—	none	—	—	—
`protocol_fee_balances_for_testing`	public	—	—	none	—	—	—

`blast_presale::blast_presale_package`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`package_witness`	package	—	—	none	—	—	—
`init_for_testing`	public	—	—	none	—	—	—
`version_for_testing`	public	—	—	none	—	—	—
`package_admin_for_testing`	public	—	—	none	—	—	—

`blast_presale::creation_policy`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`set_quote_allowed`	public	yes	—	ListingAdmin	—	—	—
`schedule_venue`	public	yes	—	VenueAdmin	—	—	—
`apply_venue`	public	yes	—	none	—	—	—
`cancel_venue`	public	yes	—	CancelAdmin	—	—	—
`remove_venue`	public	yes	—	VenueAdmin	—	—	—
`add_partner`	public	yes	—	FeeAdmin	—	—	—
`remove_partner`	public	yes	—	FeeAdmin	—	—	—
`schedule_venue_delay`	public	yes	—	VenueAdmin	—	—	—
`apply_venue_delay`	public	yes	—	none	—	—	—
`cancel_venue_delay`	public	yes	—	CancelAdmin	—	—	—
`unpause`	public	yes	—	UnpauseAdmin<BLAST_PRESALE_PACKAGE	—	—	—
`is_guarded`	package	—	—	none	—	—	—
`assert_allowed`	package	—	—	none	—	—	—
`assert_partner`	package	—	—	none	—	—	—
`venue_delay_ms_for_testing`	public	—	—	none	—	—	—
`new_publication_for_testing`	public	—	—	none	—	—	—

`blast_presale::events`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`quote_allowed`	package	—	—	none	—	QuoteAllowed	—
`presale_opened`	package	—	—	none	—	PresaleOpened	—
`protocol_fees_changed`	package	—	—	none	—	ProtocolFeesChanged	—
`committed`	package	—	—	none	—	Committed	—
`presale_funded`	package	—	—	none	—	PresaleFunded	—
`presale_cancelled`	package	—	—	none	—	PresaleCancelled	—
`presale_migrated`	package	—	—	none	—	PresaleMigrated	—
`raise_allocated`	package	—	—	none	—	RaiseAllocated	—
`recipient_allocated`	package	—	—	none	—	RecipientAllocated	—
`claimed`	package	—	—	none	—	Claimed	—

`blast_presale::fee_policy`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`schedule_fees`	public	yes	—	FeeAdmin	—	—	—
`apply_fees`	public	yes	—	none	—	—	—
`cancel_fees`	public	yes	—	CancelAdmin	—	—	—
`protocol_fees`	package	—	—	none	—	—	—
`is_guarded`	package	—	—	none	—	—	—
`fees`	package	—	—	none	—	—	—
`raise_bps`	package	—	—	none	—	—	—
`token_bps`	package	—	—	none	—	—	—
`lp_fee_meme_bps`	package	—	—	none	—	—	—
`lp_fee_quote_bps`	package	—	—	none	—	—	—
`set_fees_for_testing`	public	—	—	FeeAdmin	—	—	—
`new_zero_fees_for_testing`	public	—	—	none	—	—	—

blast_migrators (26 functions)

`blast_migrators::blast_migrators`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`init_for_testing`	public	—	—	none	—	—	—
`version_for_testing`	public	—	—	none	—	—	—

`blast_migrators::bluefin`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`prepare`	public	yes	—	owned object	out	—	—
`migrate`	public	yes	—	none	in	—	—
`collect_fees`	public	yes	—	none	—	—	—
`settle_and_apply_fee_split`	public	yes	—	none	—	—	—
`buyback_and_burn`	public	yes	—	BuybackAdmin	—	—	—
`seed_fixed_side_for_testing`	public	—	—	none	—	—	—
`auto_lp_fixed_side_for_testing`	public	—	—	none	—	—	—

`blast_migrators::cetus`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`prepare`	public	yes	—	owned object	out	—	—
`migrate`	public	yes	—	none	—	—	—
`collect_fees`	public	yes	—	none	—	—	—
`settle_and_apply_fee_split`	public	yes	—	none	—	—	—
`buyback_and_burn`	public	yes	—	BuybackAdmin	—	—	—
`seed_fixed_side_for_testing`	public	—	—	none	—	—	—
`is_meme_coin_a_for_testing`	public	—	—	none	—	—	—
`reorder_for_testing`	public	—	—	none	—	—	—

`blast_migrators::events`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`cetus_pool_reserved`	package	—	—	none	—	CetusPoolReserved	—
`bluefin_pool_tier_recorded`	package	—	—	none	—	BluefinPoolTierRecorded	—
`cetus_pool_reserved_for_testing`	public	—	—	none	—	—	—
`bluefin_pool_tier_recorded_for_testing`	public	—	—	none	—	—	—

`blast_migrators::venue_config`

Function	Vis	Version	Pause	Auth	Value	Events	Rounding
`add_object`	package	—	—	none	—	—	—
`object`	package	—	—	none	—	—	—
`add_value`	package	—	—	none	—	—	—
`value`	package	—	—	none	—	—	—
`init_for_testing`	public	—	—	none	—	—	—

Appendix C. Checked and holding

What was attacked, per gotcha category, and why it held. Line references are to the audited snapshot (`4fe8d739f04bafd0775691795eb9f3727e984089`). "PoC" names a test in the evidence bundle's `poc/`; "stock" names an existing test in the package.

4.1 Object model and abilities

- **`store` on authority objects.** `AdminCap`, `PartnerCap`, `CurveFeeCap`, `CreatorMetadataCap`, `PresaleCap`, `LaunchTicket` carry `store` and are transferable by design (ADR 0082, 0137). Each is bound to live registry state or an exact ID (`blast_acl::sign_in`, `launch::assert_curve_fee_cap`, `blast_presale::assert_cap`), so possession alone authorizes nothing after revocation or reassignment. `SuperAdminCap`, `Publication`, `PackageVersion`, `PackageAdmin`, `Launch`, `Presale`, `Vault`, `Round` are `key`-only.
- **Hot potatoes.** `VenueTicket`, `Migration`, `OwnerProof`, `LaunchInitializer`, `PresaleInitializer`, `BuybackReceipt`, `AutoLpReceipt`, `UpgradeTicket`/`UpgradeReceipt` have no abilities. Each has exactly one consumer in the defining module; receipts bind the vault ID and cap the returned amounts at the taken amounts (`vault.move:363-394,422-452`; stock `a_buyback_receipt_from_another_vault_is_rejected`, `an_auto_lp_receipt_from_another_vault_is_rejected`).
- **`copy` on value.** `VaultTerms` and `FeeSplit` are `copy`, but neither authorizes anything: `vault::new` admits only terms whose sealing witness is on the registry and only through an admitted venue's witness value, and venues take terms only from `migration::redeem` (`vault.move:118-172`, `cetus.move:123`, `bluefin.move:89`; stock `terms_sealed_by_an_unadmitted_issuer_cannot_create_a_vault`).
- **Object substitution.** Every configuration object is a singleton created only in `init` (`LaunchConfig`, `RuntimeConfig`, `Treasury`, `VaultRegistry`, `LaunchRegistry`, `PresaleRegistry`, `CreationPolicy`, `FeePolicy`, `VenueConfig`, `ACL`); `PackageVersion<OTW>` and `PackageAdmin<OTW>` exist once per OTW because `wrap` consumes the unique `Publication`. Caps check IDs (`curve_fee_cap_id`, `creator_metadata_cap_id`, `presale_cap_id`, `super_admin_cap_id`); rounds and receipts check `vault_id`; venue collections check `pool_id`.
- **Type confusion.** `Meme != Quote` is enforced at `launch::new`, `protocol_fees::new` and `presale::new` by `with_original_ids`. Allowlists (venues, Quotes, vault witnesses) compare `with_original_ids`, which survives upgrades; roles compare `with_defining_ids`, which is stable for types first defined in a package's original version. The migrators retype balances between coin orders through a dynamic-field round trip that aborts in the framework on any type mismatch (`cetus.move:542-565`; stock `a_reorder_to_another_type_aborts_in_the_framework`).
- **Admission filters.** `schedule_witness`/`schedule_venue` reject primitives and framework types only. A `drop` type with a public constructor outside 0x1/0x2 (`Bps`, `FeeSplit`, `Recipient`, `VaultTerms`) would pass the filter; admission is a reviewed administrative decision and the docs say so (`creation_policy.move:60-63`). Not a code defect.
- **Regulated coins.** Sale tokens: the launchpad creates the currency itself (never regulated); the presale requires `Unregulated` and `Unknown` supply by reading the `Currency` BCS prefix, whose layout matches the pinned framework (`coin_registry.move:96-148`: `SupplyState::Unknown` = tag 2, `RegulatedState::Unregulated` = tag 1). Quotes: USDC on mainnet is `Regulated` (deny cap `0x699b…0ffe`); deny-listed recipients blocking payouts, collection and re-splits are accepted (ADR 0134, PS-I3). No `TreasuryCap` survives `finalize` in either product; `vault::burn` sends `Fixed`-supply Meme to `0x0` and burns `BurnOnly` Meme.
- **Coin and balance handling.** Zero inputs are rejected or no-ops (`zero_opening_round_bid`, `zero_commitment`, `zero_trade_output`); refunds and residuals are returned or vaulted; collection balances end at exactly zero (`destroy_zero`, `vault.move:305-306`).
- **Dynamic fields and tables.** Bid and commitment keys are per address; claims remove them; presale participants ≤ 65,536; `DelayedMap` has no consumer at this commit; `PartnerList` tables are admin-bounded; `VenueConfig` entries are write-once per Meme and never removed.
- **`init` and bootstrap.** Each upgradeable package's `init` only mints the `Publication` (and id-only registries / zero-delay policies); `wrap` consumes it with a version-1 cap (stock `wrap_rejects_an_upgrade_cap_for_another_package`). The two immutable core singletons (`Treasury`, `ACL`) abort on a `0x0` address at publish.
- **Wrapped objects and receive.** Positions are dynamic object fields (discoverable by ID); the `PoolCreationCap` likewise. Payouts to caller-supplied addresses: opening-round bidders and presale participants are senders; recipients and routes are creator- or owner-chosen; holder-reward leaves are off-chain-chosen (finding I-3).
- **Address balances.** No path reads or writes an address balance; both creation fees are explicit coins.

4.2 PTBs and atomicity

- **Same-transaction composition.** No check assumes "not in the same transaction": pre-share buys are deliberate, delayed proposals compare `now ≥ ready_at`, and the opening round cannot be bought or sold before settlement. A creator can buy to graduation and migrate before `share`; nothing is taken from anyone (architecture.md "a buy before sharing may do this").

- **Clock constancy.** Only `≥`/`<` deadline comparisons exist; no per-timestamp counters.
- **Hot potatoes across objects.** `Migration` carries the stored venue and the sealing witness; `redeem` checks the venue (`migration.move:136-146`); `OwnerProof` checks witness and origin; receipts check the vault. Stock tests cover each wrong-object case.
- **Permanent public surface.** No public function returns a raw `Balance`/`Coin` of protected state except owner and creator claims to their own balances, and venue takes that return receipts. `treasury_cap_mut` no longer exists; `publisher_mut` returns `&mut Publisher`, which the pinned framework cannot move out of, and needs the package witness.
- **Sponsorship.** `ctx.sender()` is used for bids, commitments, claims and the creator-claim claimant; none depends on the gas coin.
- **Randomness.** None used.

4.3 Arithmetic

- **Shifts.** `sqrt_price_x64`: `(u64 as u256) << 128` cannot truncate; the binary search bound `2^96` is proven by the comment and the PoC `audit_poc_seed_sqrt_price_is_the_exact_floor_square_root`. `holder_rewards::claim_bit`: `1u256 << (index % 256)`. Bluefin side selection: `amount << 64` and `liquidity << 64` with `liquidity < 2^128` checked (`bluefin.move:407-426`). No custom checked-shift helper exists.
- **Overflow and abort-as-DoS.** Sums of one coin's amounts fit u64 by supply; `total_commitments` uses `checked_add`; deadlines use `checked_add` at `share`/`publish_round`; apportionment widens to u128; curve math widens to u256 and the output never exceeds the output reserve. A sell whose gross output exceeds the real reserve aborts in `Balance::split` (framework code, accepted LP-I4); PoC `audit_poc_round_trip_and_split_buys_never_profit` finds no such case over 400 random buy–sell pairs.
- **Rounding.** Fees and Blast's shares round up (`apply_up`, `mul_div_ceil`, `apportion` rounded-up index); user outputs, pro-rata deliveries, refunds and partner cuts round down. PoC `audit_poc_apportionment_conserves_and_rounded_up_part_gets_its_ceiling` (300 random vectors, every rounded-up choice) and `audit_poc_capped_crossing_net_equals_remaining_gap` (400 cases).
- **Zero outputs.** A dust buy returns its input whole unless it graduates (`launch.move:988`); a sell with zero net aborts; splitting trades never beats one trade (PoC).
- **Graduation crossing.** Capped net input equals the remaining gap exactly, so `reaches_target` and `real == target` coincide and `migrate` apportions exactly the target (PoC; algebra in the report).
- **Divide by zero.** Curve denominators are `V + R + in ≥ V > 0` and `Rm + in` with `Rm > 0` always (output is strictly below the reserve); deliveries divide by the bidder's own positive bid total; `apportion` divides by 10,000; `reaches` multiplies in u128.
- **Fixed point and ticks.** Seed price bounds are checked by the presale at creation and after rate changes (`assert_seedable_price`); the launchpad does not check them, but with admitted decimals 6 or 9 and a one-billion supply the terminal price `lp_quote / lp_meme` is seedable for every target of at least a few Quote atoms (`lp_meme ≤ 10^18`, `min_sqrt·2 → price ≥ 2.2·10^-19`), so only a target below three atoms could strand a launch, and only its creator's own atoms. Full-range ticks come from the venues' own helpers.
- **Caps.** Trade fees ≤ 1,000, protocol share ≤ 7,000, allocation ≤ 1,000 each, LP shares ≤ 3,000 each, partner ≤ 8,000, creation fee ≤ 10 SUI, sell burn ≤ 5,000, presale raise/token ≤ 1,000: enforced in each stored value's constructor and re-checked at consumption in the presale (`finalize`, `set_protocol_fees`).
- **Time units.** Milliseconds everywhere; `period_duration_ms · periods ≤ 2^63`; vesting `start_ms = now`.

4.4 Swaps and DEX integration

- **Slippage.** `buy` and `sell` apply `min_*_out` to the net amount after both fees; buybacks apply `min_meme_out` to the Meme actually returned (`vault.move:439`). Permissionless collections carry no price bound by design; the only price-sensitive step is the auto-LP add, bounded by one collection's fee share (finding I-4 notes the coupling).
- **Spot price as oracle.** No value-bearing decision reads a pool price; the seed price comes from the hand-off balances.
- **Pool griefing.** Cetus creation is reserved at `prepare` with the Meme's single `PoolCreationCap` before any Meme leaves custody; migration creates the pool with that cap at the recorded spacing and adopts nothing. Bluefin pools are not registry-unique, so a pre-created pool cannot collide. A Cetus pool-manager override remains a venue-operator assumption (MG-I1, accepted).
- **Coin order.** Cetus routes take `<CoinA, CoinB>` and prove they are exactly `{Meme, Quote}` in Cetus's order (`is_right_order`, `is_meme_coin_a`); Bluefin always uses `Pool<Meme, Quote>`. Both orders are in the fork matrix (DEPENDENCIES.md).
- **LP positions.** The Cetus position is burned and the vault holds the `CetusLPBurnProof`; the auto-LP position is a dynamic object field with no removal path; Bluefin's position likewise. Fee claims route only through `vault::split_fees`.
- **External gates.** Cetus `GlobalConfig` and Bluefin `GlobalConfig` version checks happen inside the venues; a venue pause or bump aborts migration and collection until a migrators relink (accepted MG-I1). `prepare` fails closed on an unregistered Cetus tier.
- **Keepers.** Collections and `settle_and_apply_fee_split` are permissionless and per vault, so one failing vault blocks nothing else. Buybacks need `BuybackAdmin`.
- **Recipients and partner share.** No caller supplies a recipient: treasury from `Treasury`, partner from the bound `Partner` object, routes from the applied split, residuals by the sealed policy. The partner cut is taken exactly once per ledger payout (`protocol_fees.move:100-116`) and nowhere else.
- **Interface fidelity.** `scripts/check_mainnet_abi.mjs` passed live on 2026-11-05 (5 packages, 11 modules, 38 functions, 14 structs, module fingerprints unchanged). Independently fetched from `graphql.mainnet.sui.io`: `pool_creator::create_pool_v3_with_creation_cap` (13 parameters, returns `(Position, Coin<A>, Coin<B>)`), `lp_burn::burn_lp_v2` (no type parameters), `lp_burn::collect_fee` and Bluefin `pool::create_pool_and_get_object`, `add_liquidity_with_fixed_amount`, `swap` match the stubs' parameters, references, type-parameter counts and returns. Pinned `published-at` addresses match the live current revisions (Cetus v15 `0x2606…`, LP Burn v8 `0xc4aa…`, Bluefin v17 `0xd075…`).

4.5 Authority, admin and upgrades

- **Cap binding.** Every creator-cap use checks the recorded ID; every role check is a fresh `AdminWitness<Role>` from the live ACL; revocation (`revoke_admin`, `revoke_role`) is immediate and a destroyed or revoked cap signs in to nothing (stock ACL tests).
- **Version gates.** Enumerated in function-inventory.md: every state-mutating public function of the four upgradeable packages calls `assert_current` first, with two deliberate exceptions (`vault::share_with_position`, reachable only in the same transaction as the gated `vault::new`; `blast_vault_package::unpause`, finding L-1). The immutable `blast_admin` and `blast_core` have no consumer gate by design.
- **Delayed settings.** Ready time uses the applied delay; a shorter proposed delay waits the old one; replacements restart; a notice above 30 days aborts; `take_action` and `apply_delay` reject the other kind; `is_guarded` requires nonzero applied delay, no pending zero and (sets) no pending admission. Overwriting a pending change cancels it with an event. `&UID` owner binding keeps keys and events attributable (stock delayed tests, 198 admin tests).

- **Pause.** Products: create/trade-or-commit/migrate bits; claims, refunds, cancels, deliveries, settlement never pause (except opening-round settlement under the trading bit, accepted LP-I3). Vault: admin bit only. Migrators: none (accepted residual, ADR 0137 amendment). Unpause needs the package witness, the guarded upgrade delay and each product's guarded configuration delays.
- **Upgrades.** The cap never leaves `PackageAdmin`; a scheduled exact digest waits the applied delay (48 h at wrap); the receipt commits into the same cap and advances the version in one call; `make_immutable` is a scheduled action that also needs the package witness and, in the vault, the current version.
- **Init parameters.** `blast_acl_root` and `blast_treasury` abort at `0x0`; the fixture addresses used for tests are the CI ones.
- **Documented power.** No role exceeds its ADR: the remaining instant powers (root grants, pause, immediate venue and signer removal, 30 % LP-share ceilings) are the accepted trust boundaries of the ADR 0134 amendments.

4.6 Protocol and economic logic

- **Presale.** Caps, minimums and participant bounds hold under every phase (`record_commitment`); a hard-capped fill accepts exactly the remainder and allows the filling commitment below the minimum; early close recomputes the LP deadline from the close; refunds are full on cancellation and pro rata on overflow; the last claim takes the remainder; `set_protocol_fees` only lowers rates once shared and re-asserts the seedable price; team shares vest from migration.
- **Merkle.** Leaves are `blake2b(0x00 || domain || payload)` and nodes `blake2b(0x01 || l || r)`; holder leaves bind vault, round, index, beneficiary and both amounts; allowlist leaves bind index, wallet and cap; proofs are positional with exact depth and `index < leaf_count`; a claimed-index bitmap stops replays (PoC `audit_poc_merkle_rejects_wrong_depth_index_order_and_sibling`).
- **Signatures.** The creator claim signs `b"blast_fun::creator_claim::v1" || BCS{meme, commitment, claimant, expiry}`, checks `now ≤ expiry`, verifies against any live signer key, and is consumed once by extracting the creator recipient.
- **Strict equality.** The presale checks `creation_fee.value() == config.creation_fee()` on a caller-supplied coin (exact payment, not a balance invariant). No `balance == recorded` check exists elsewhere; donations into a shared object's balance are impossible without a function.
- **Sybils.** Per-wallet caps are a sale-design feature, not a security property.
- **Spam.** Creating a launch costs the creation fee and a coin package; a presale needs a coin; both are per-coin unique. `cetus::prepare` is permissionless for any coin's cap holder and writes one entry per Meme.
- **Vault (ADR 0137).** One witness list gates venue and issuer; venues never take terms from callers; receipts bind each handoff; the only withdrawers are the owner (own balances, with a proof), Blast (its ledger, to the treasury) and the burn route.

4.7 Bounded work and liveness

- Recipient lists ≤ 16 (launch) and ≤ 64 (presale, each list); routes ≤ 20; signer keys ≤ 4; roles per admin ≤ 32; reward leaves ≤ 65,536 (bitmap of 256 words); presale participants ≤ 65,536 but every per-wallet operation is one call. The largest single transaction is a presale migration with 128 payouts, 128 events and up to 128 vesting objects, within Sui's per-transaction limits.
- Hot shared objects: each launch, presale, vault and round is its own object; registries are written only at creation; `LaunchConfig`/`RuntimeConfig` are read on every trade but written only by delayed administration.
- Exits: Live launches can sell; funded presales can be cancelled permissionlessly at the LP deadline; rounds return funds at the deadline; owner and creator claims never pause. Accepted stuck states: `MigrationReady`/`OpeningRound` under pause (LP-I6, LP-I3), a cancelled presale's tokens.

4.8 Events and off-chain coupling

- Every value movement emits an event with post-fee amounts and the entity ID: `Bought`/`Sold` (fees split and reserves after), `CreatorFeesClaimed`, `ProtocolFeesPaid` (treasury and partner amounts), `MigrationAllocationPaid`, `ProtocolFeesAccrued`, `Migrated`, `OpeningRoundJoined`/`Settled`/`Delivered`, `Committed`, `PresaleFunded`/`Cancelled`/`Migrated`, `RaiseAllocated`, `RecipientAllocated`, `Claimed`, `VaultCreated` (terms and residuals), `FeesCollected` (gross, Blast's cut, each route), `AutoLiquidityAdded`, `BuybackBurned`, `OwnerBalancesClaimed`, `RoundPublished`/`RewardClaimed`/`GasReserveSettled`/`RoundClosed`, `PartnerPaid`, and the delayed-primitive lifecycle events keyed by owner and key. Gap: finding I-2.
- Error registries: 337 first-party errors pinned by exact-abort tests (`check_move_errors.py`); each package's codes are sequential from zero.
- blast-infra decoders on `origin/main` (7c52521) target the contracts #168 layouts; the #198–#212 module and event moves are not decoded yet (issue filed; see the report's Coverage and Limitations).

4.9 Test-suite gaps (not security findings)

- No vault test asserts that `unpause` rejects a stale version; launchpad and presale have `stale_version_is_rejected_before_the_delay_gate`.
- No launchpad test pins the seedable-price bound of a launch's terminal price; the presale has `unseedable_price` tests.
- The migrators' Move suite (40 tests) covers only pure helpers, `prepare` and the registry; every venue interaction lives in the Rust fork gate, which this audit did not build.
- No test covers a holder-reward leaf whose beneficiary is an object address (I-3); the behaviour is a framework transfer, so a unit test cannot observe the stranding.
- No PTB-level test was run here (the `ptb_tests` harness needs a full simulator build).